

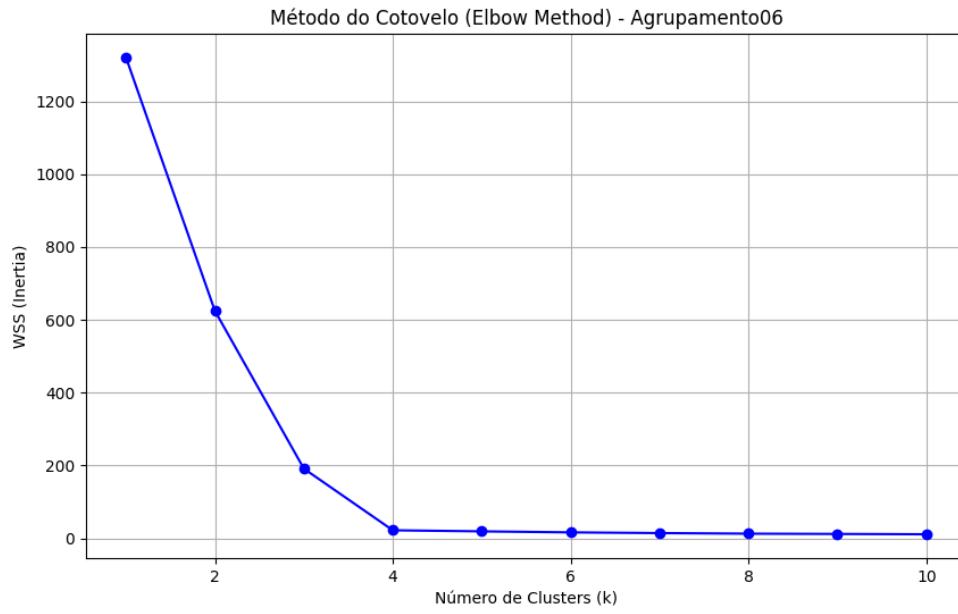
Terceiro Trabalho Prático sobre Agrupamento

Aluno: Pedro Vilaça

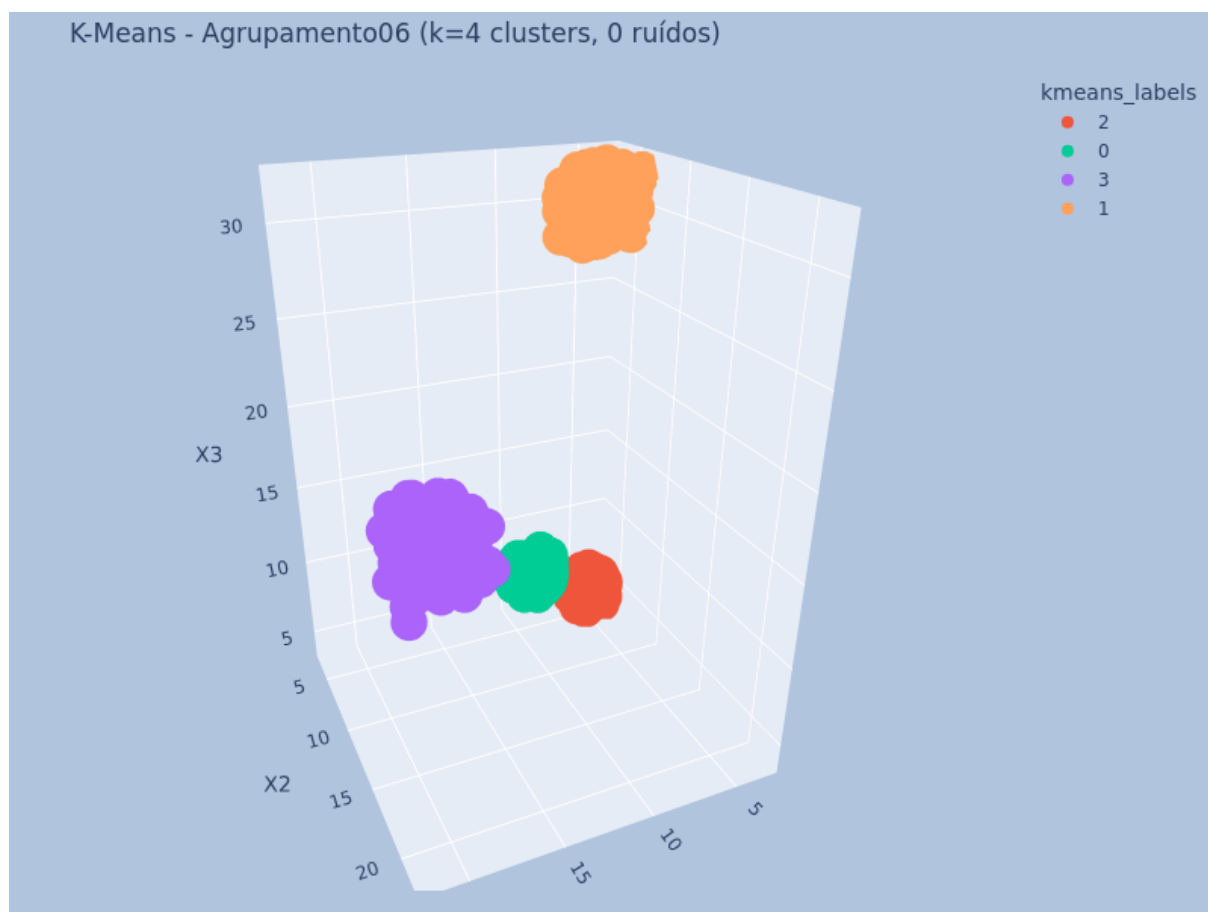
Disciplina: Inteligência Artificial

Problema 1: Agrupamento06.txt

Método do Cotovelo (Elbow Method)

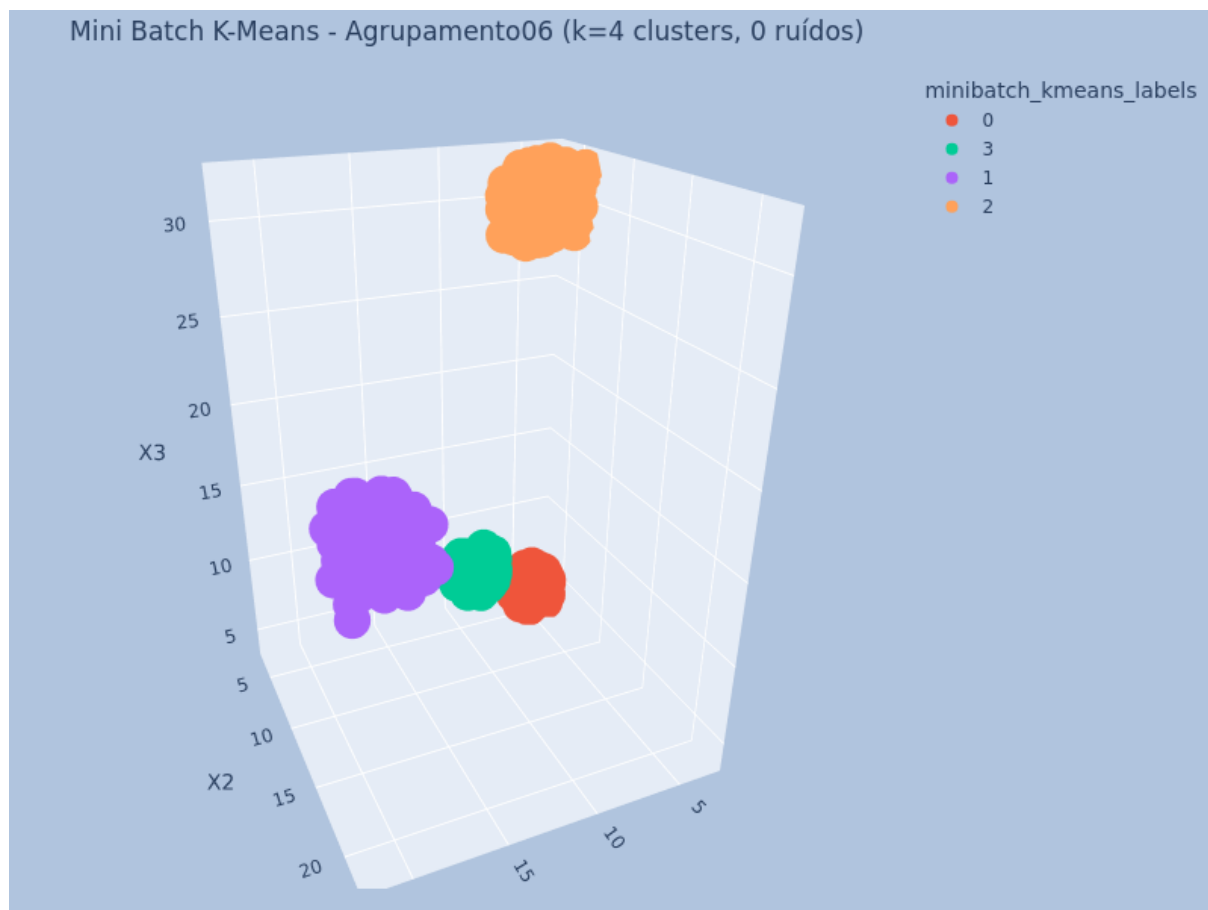


1.1) K-Means



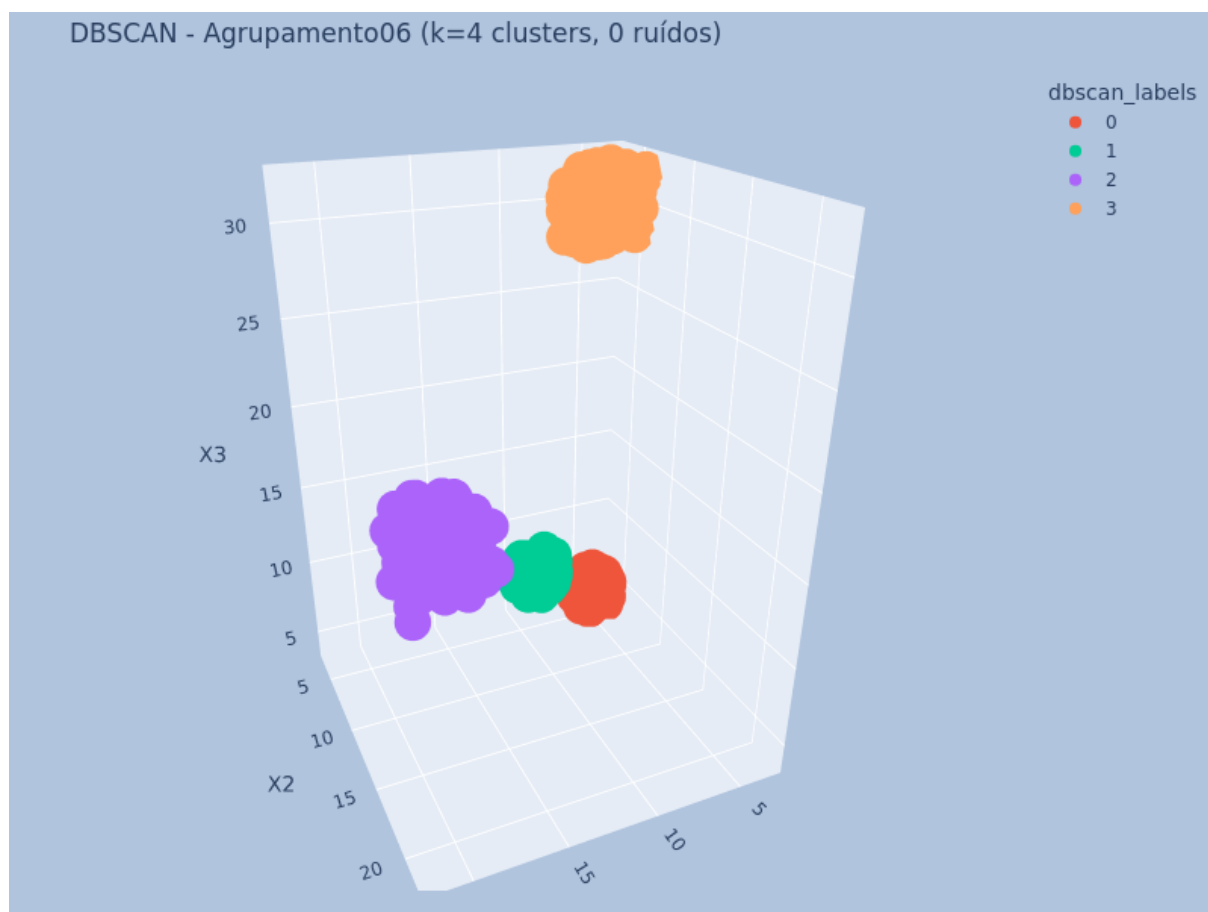
[Abrir versao interativa \(p1_kmeans.html\)](#)

1.2) Mini Batch K-Means



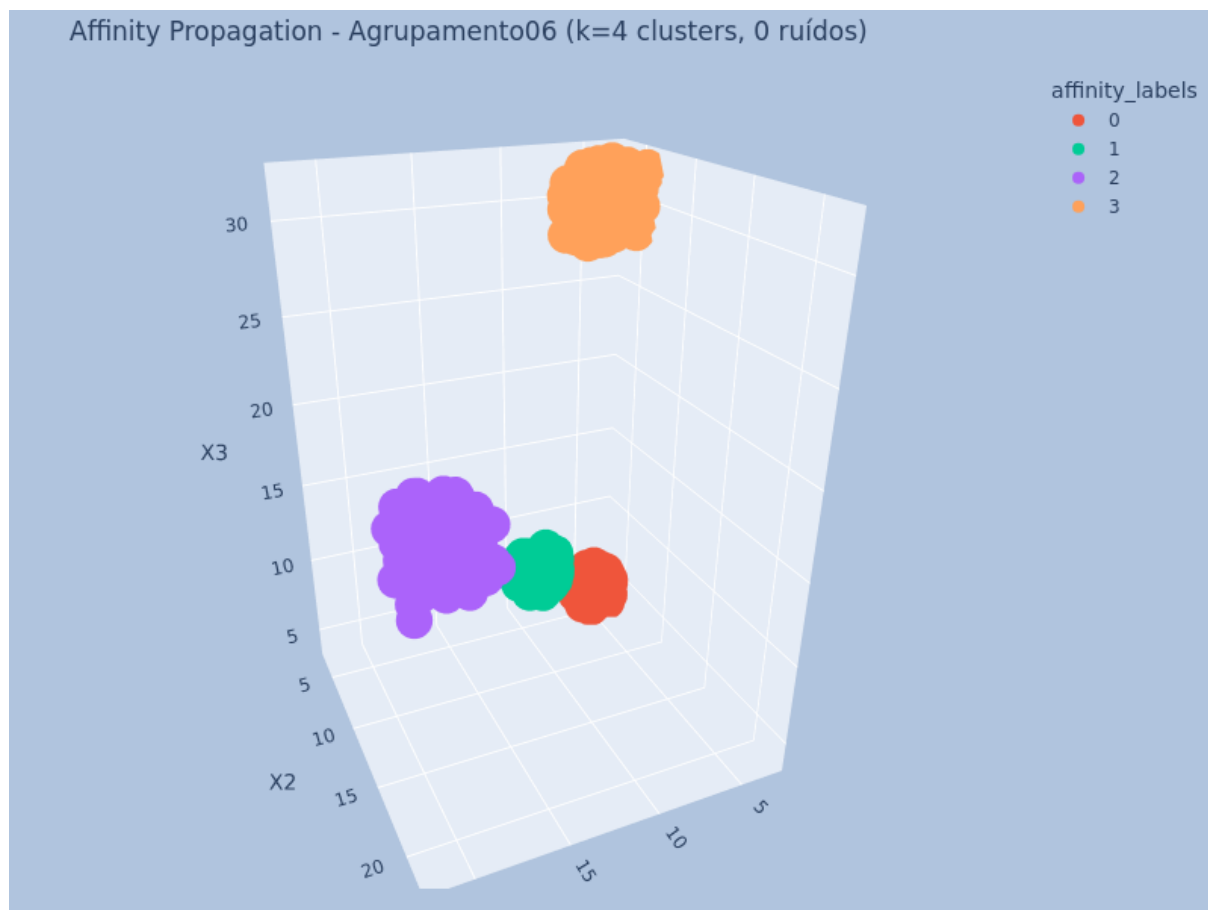
[Abrir versao interativa \(p1_minibatch_kmeans.html\)](p1_minibatch_kmeans.html)

1.3) DBSCAN



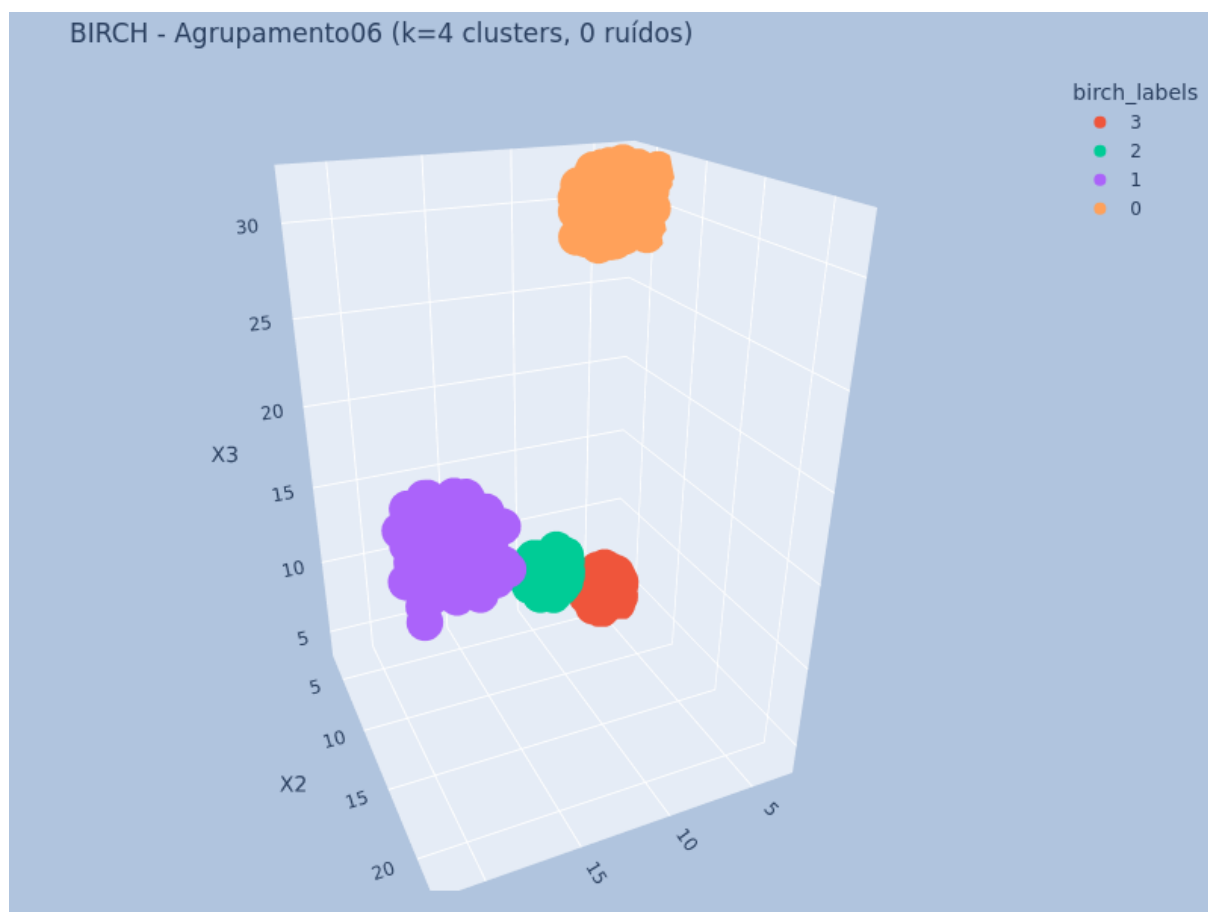
[Abrir versao interativa \(p1_dbscan.html\)](#)

1.4) Affinity Propagation



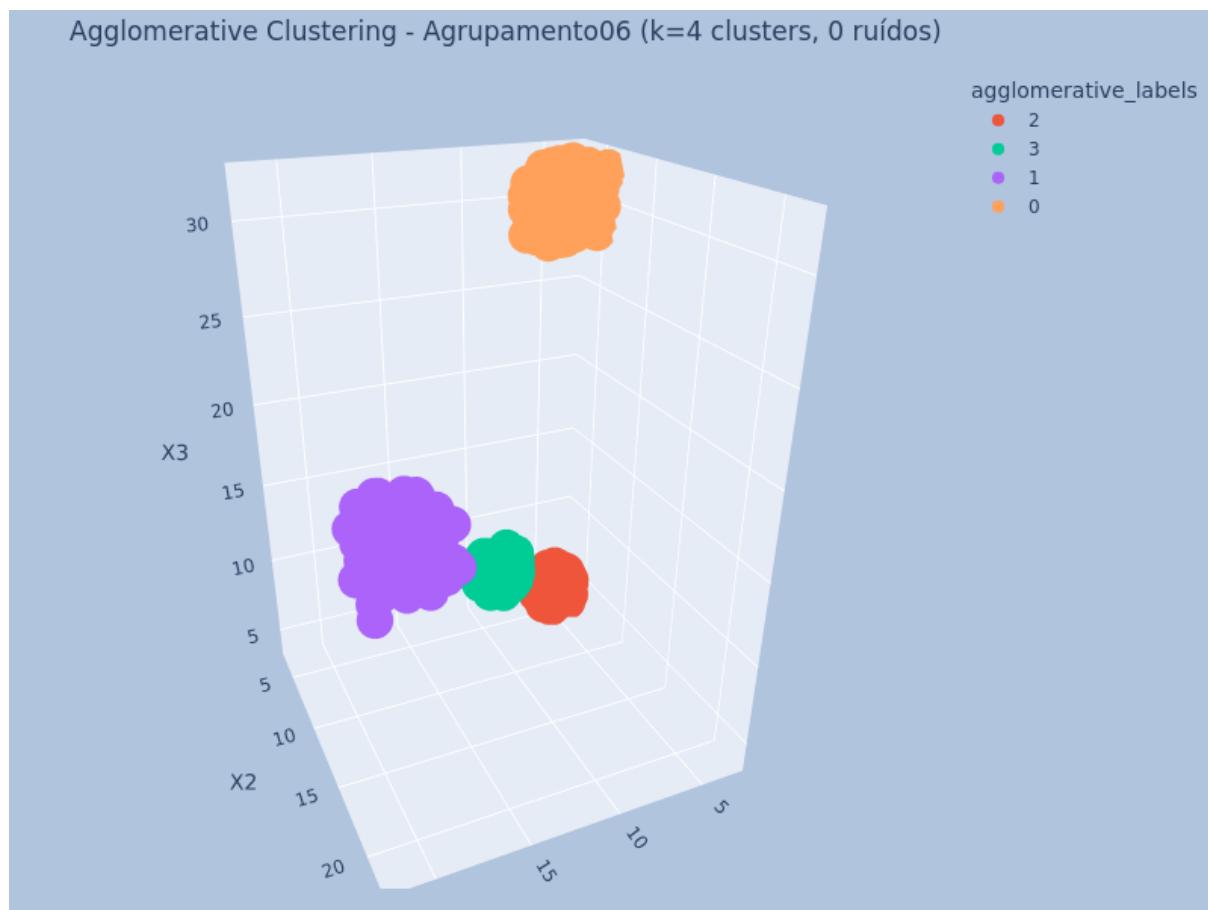
[Abrir versao interativa \(p1_affinity.html\)](#)

1.5) BIRCH



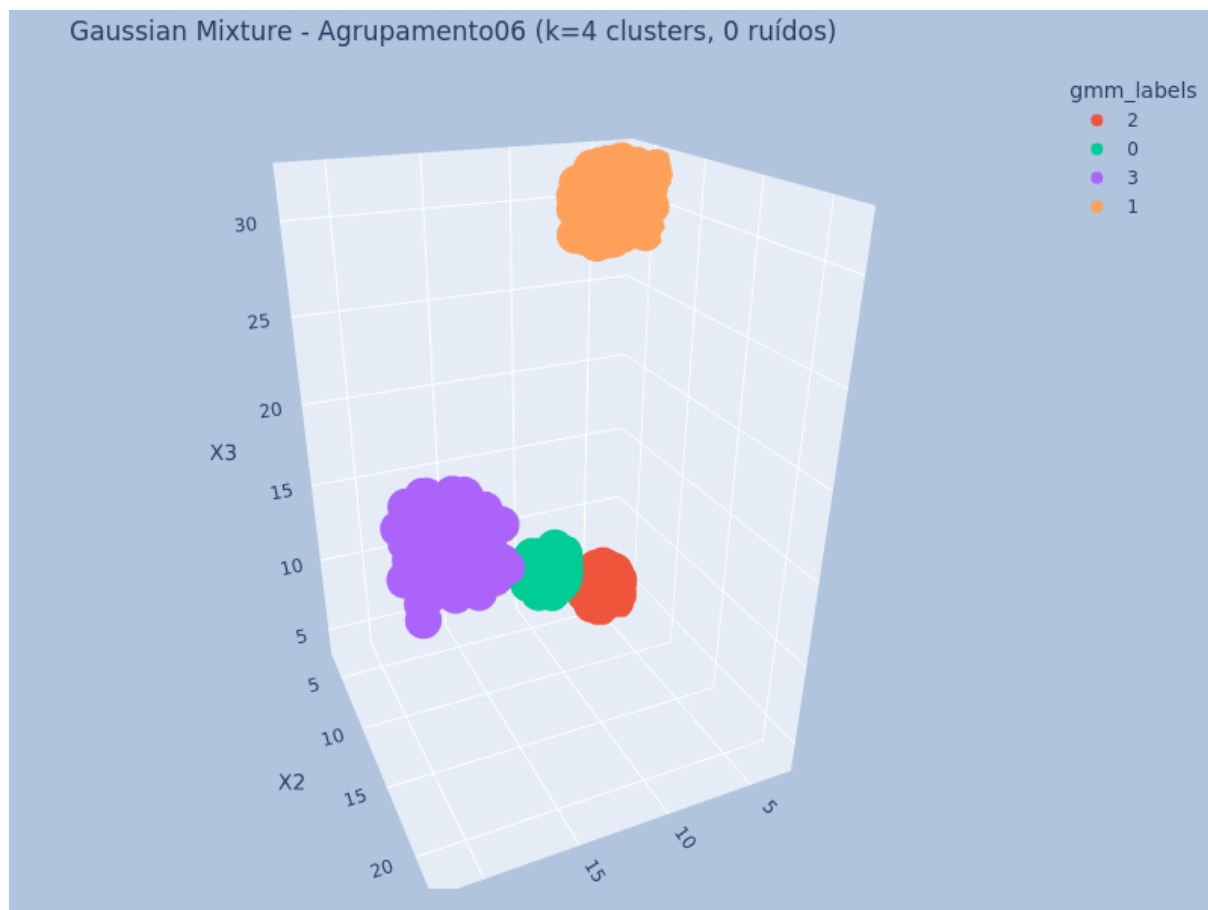
[Abrir versao interativa \(p1_birch.html\)](#)

1.6) Agglomerative Clustering



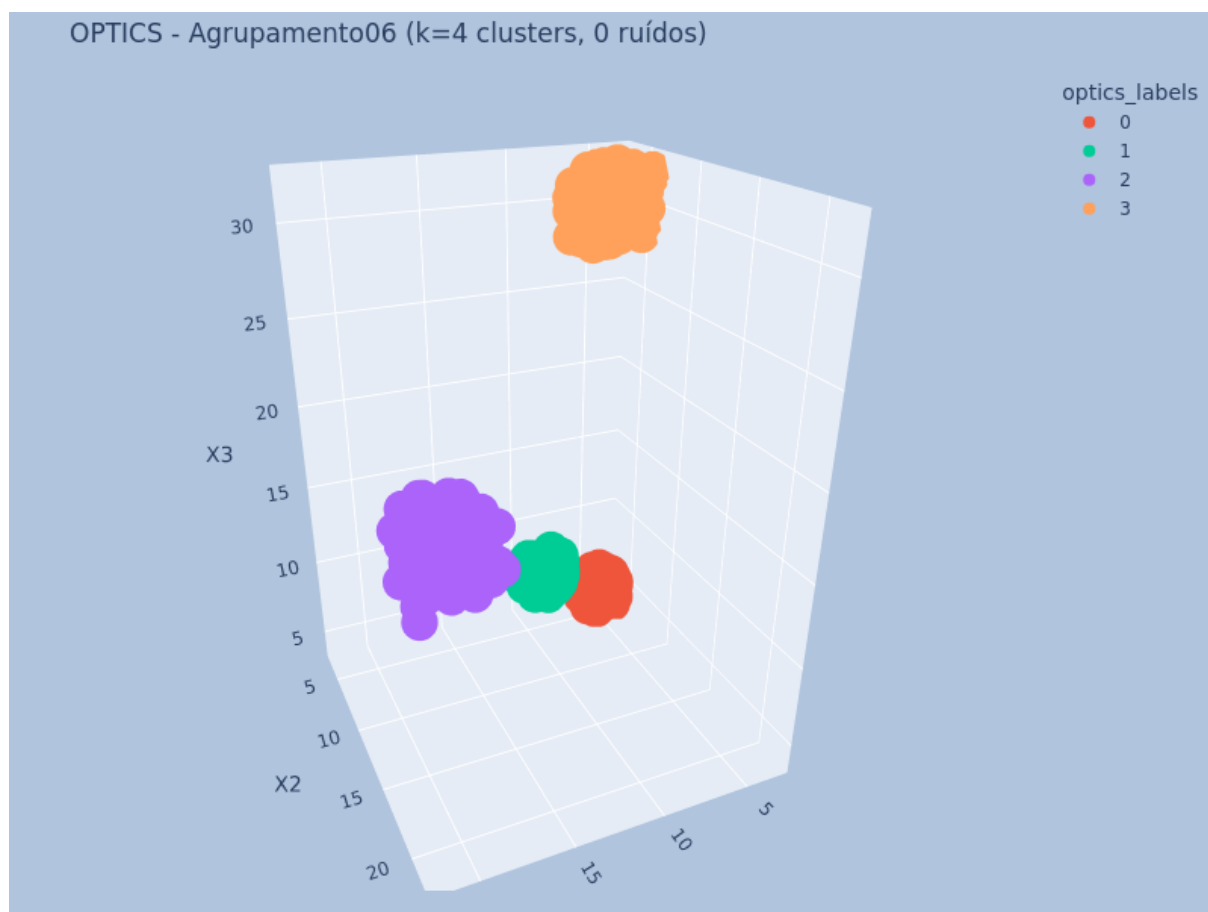
[Abrir versao interativa \(p1_agglomerative.html\)](#)

1.7) Gaussian Mixture



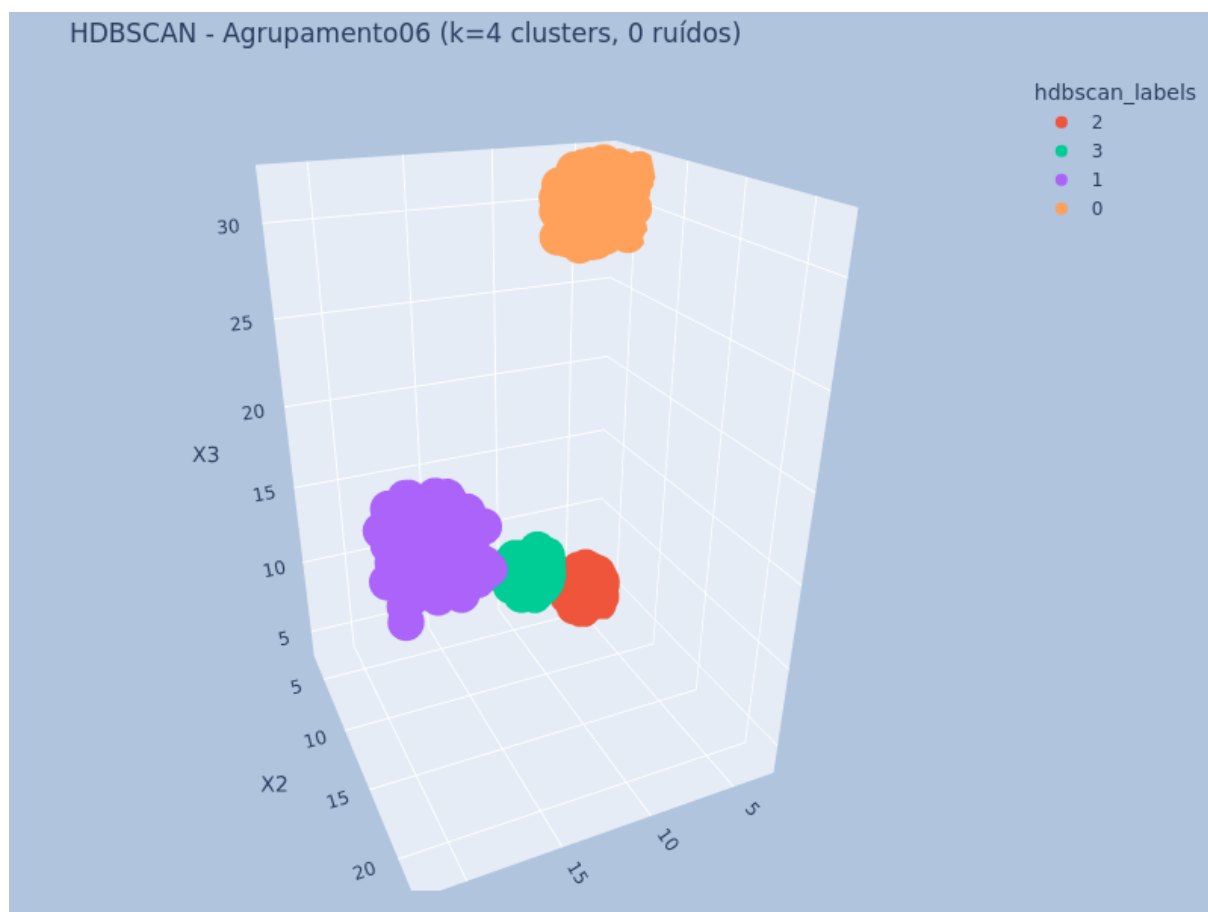
[Abrir versao interativa \(p1_gmm.html\)](#)

1.8) OPTICS



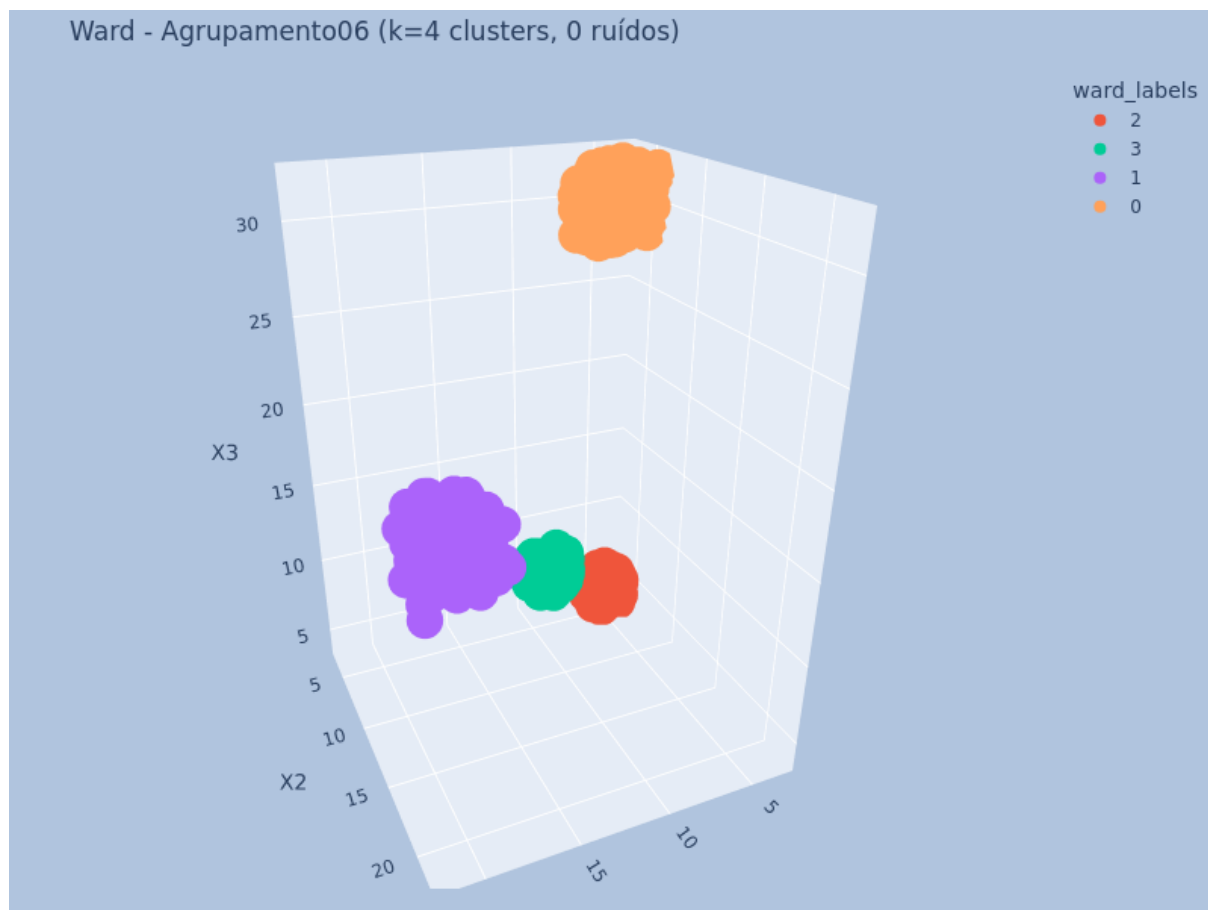
[Abrir versao interativa \(p1_optics.html\)](#)

1.9) HDBSCAN



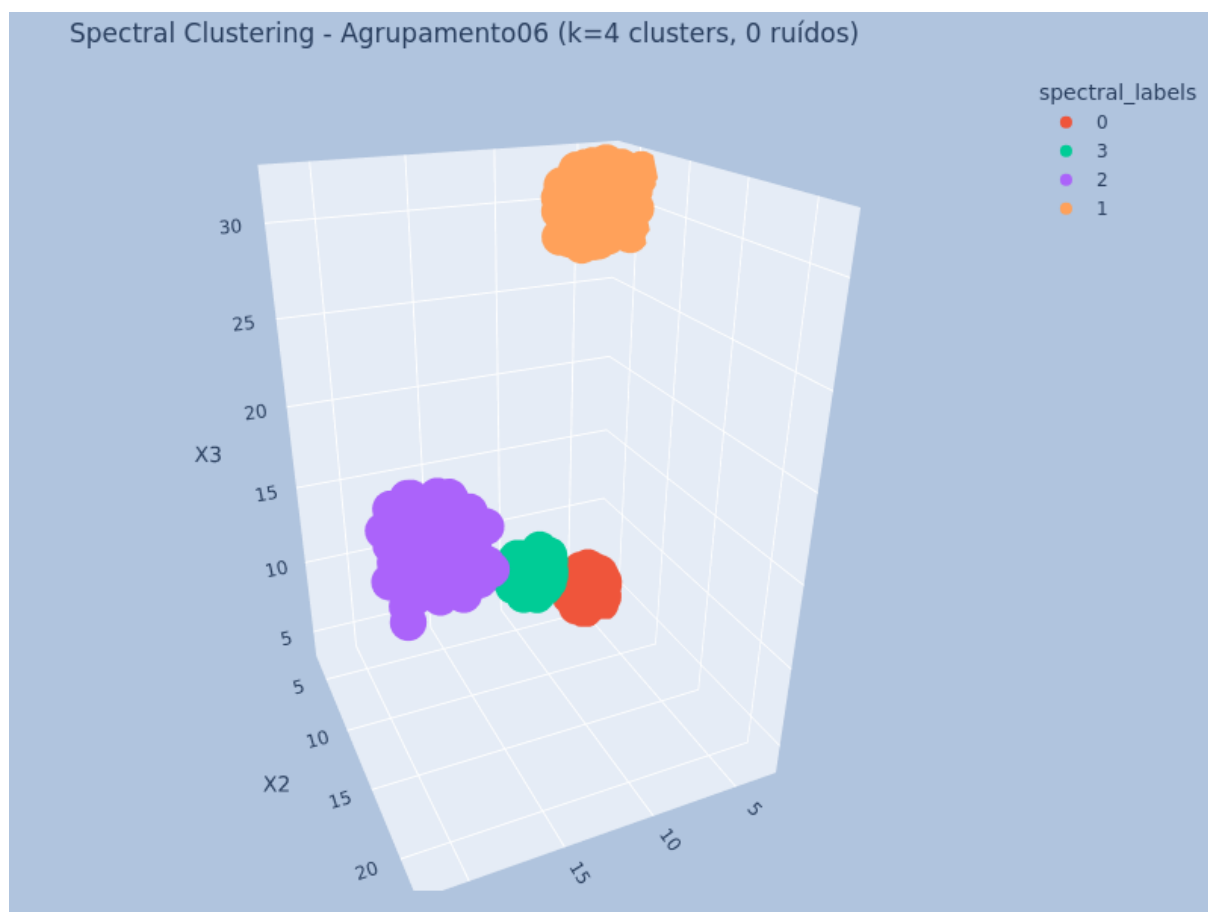
[Abrir versao interativa \(p1_hdbscan.html\)](#)

1.10) Ward



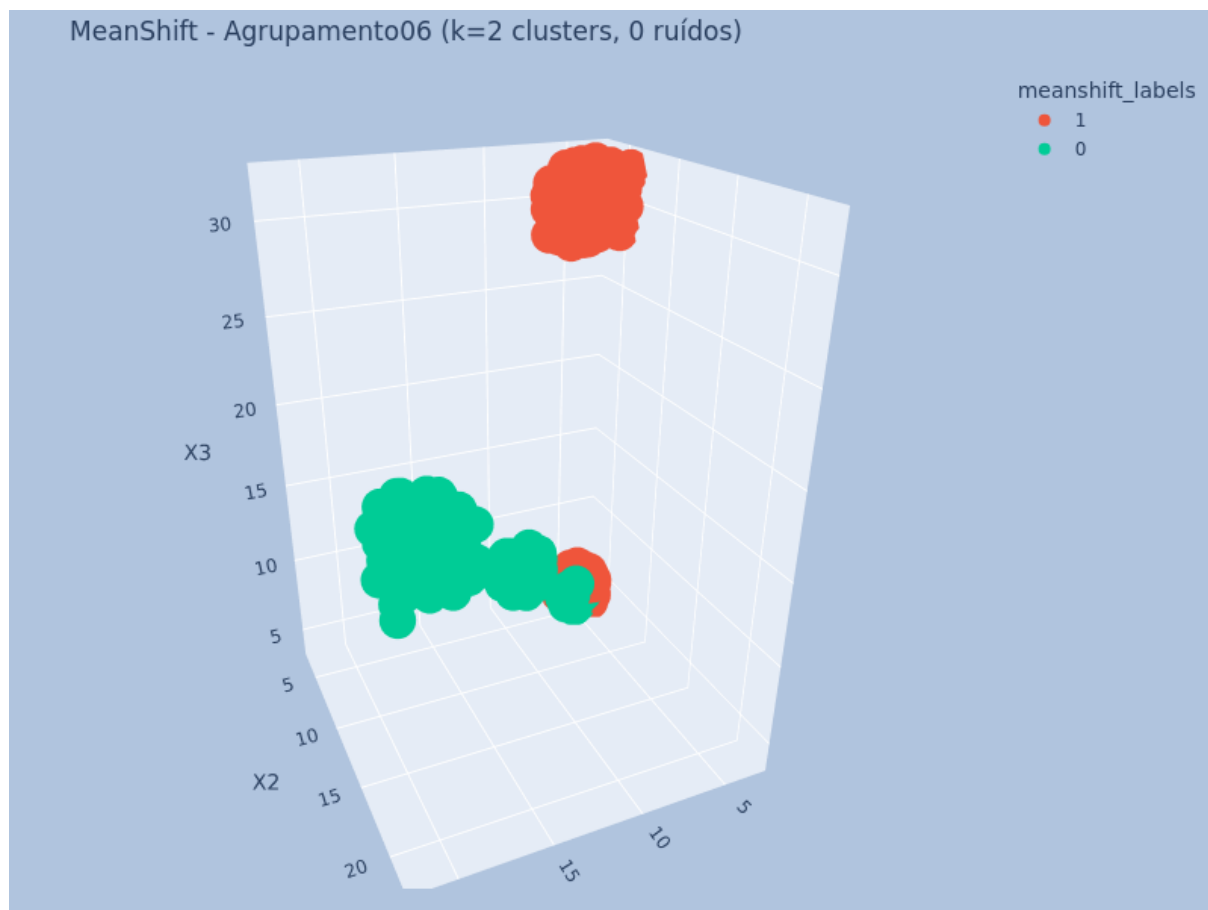
[Abrir versao interativa \(p1_ward.html\)](#)

1.11) Spectral Clustering



[Abrir versao interativa \(p1_spectral.html\)](#)

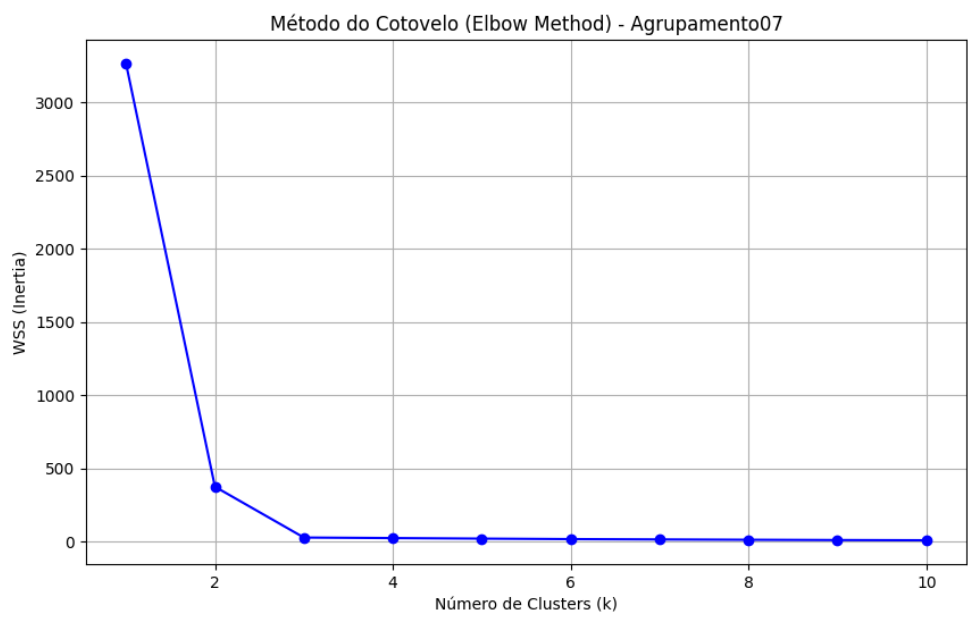
1.12) MeanShift



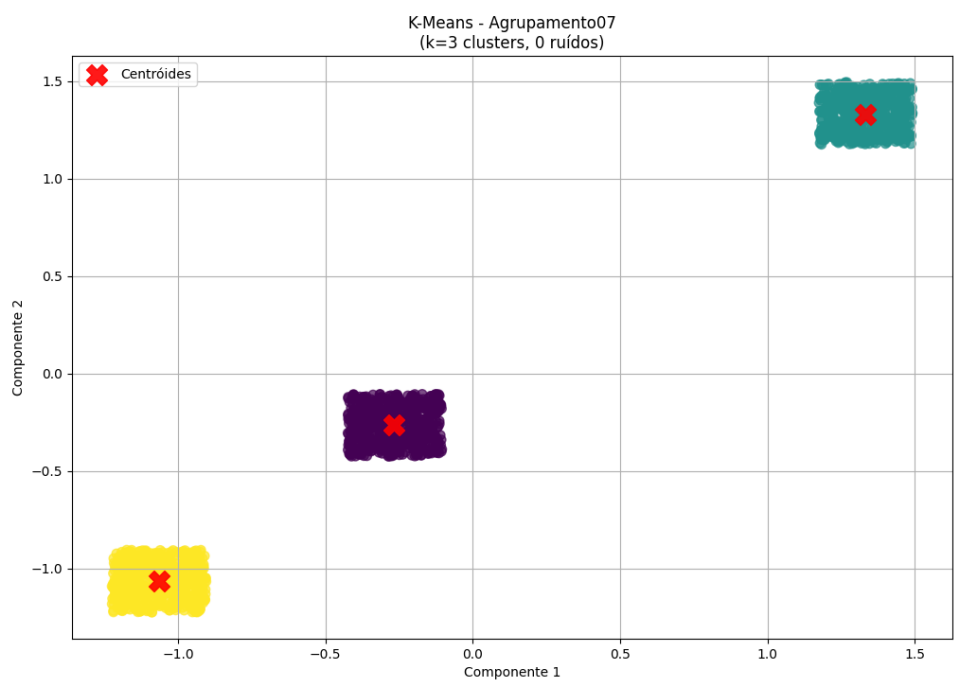
[Abrir versao interativa \(p1_meanshift.html\)](#)

Problema 2: Agrupamento07.txt

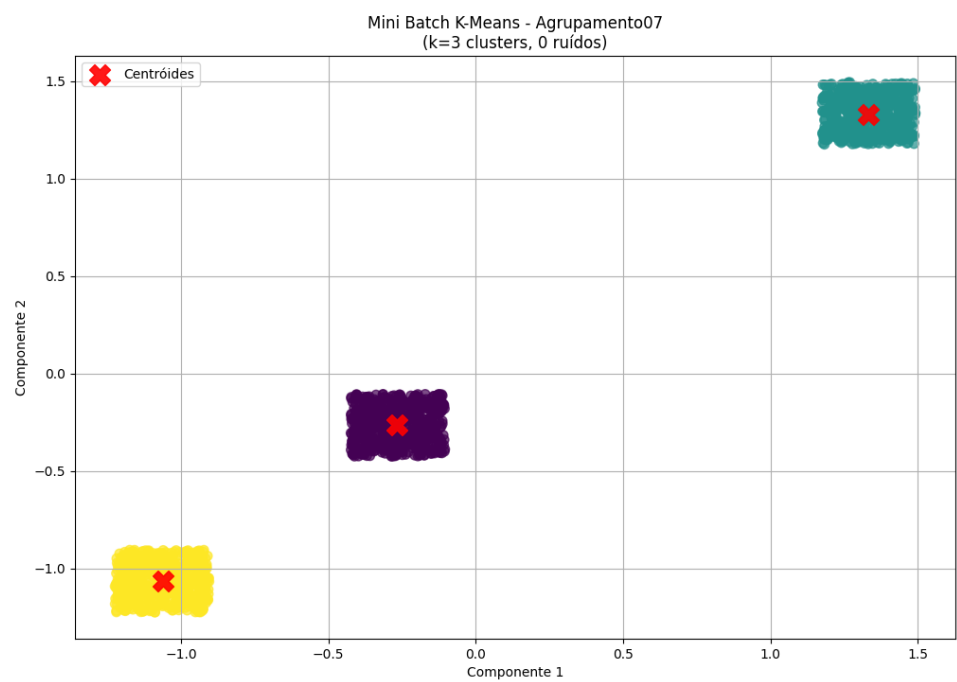
Metodo do Cotovelo (Elbow Method)



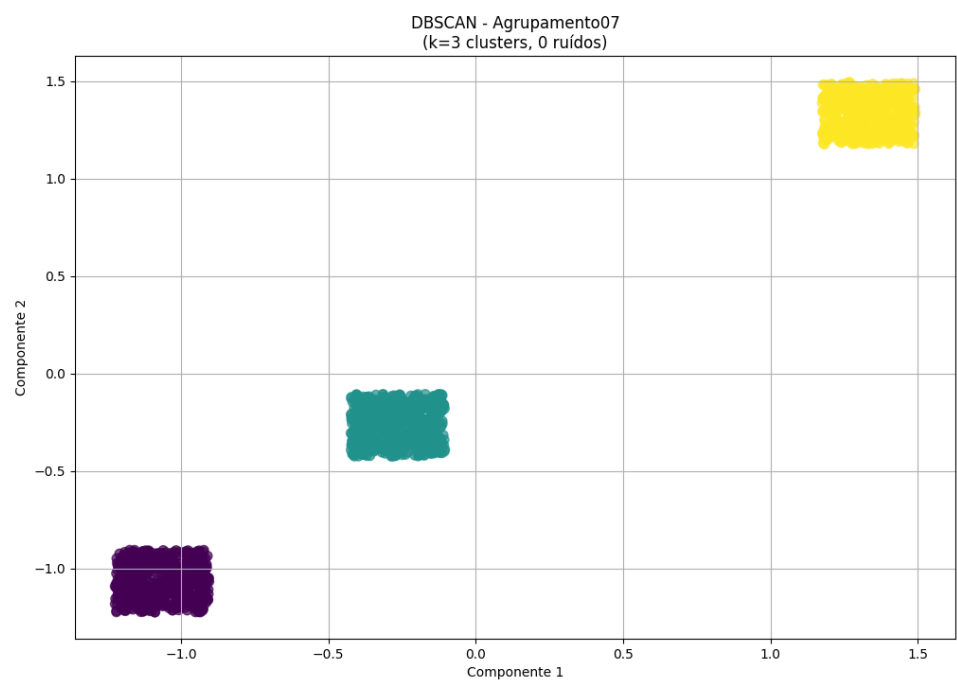
2.1) K-Means



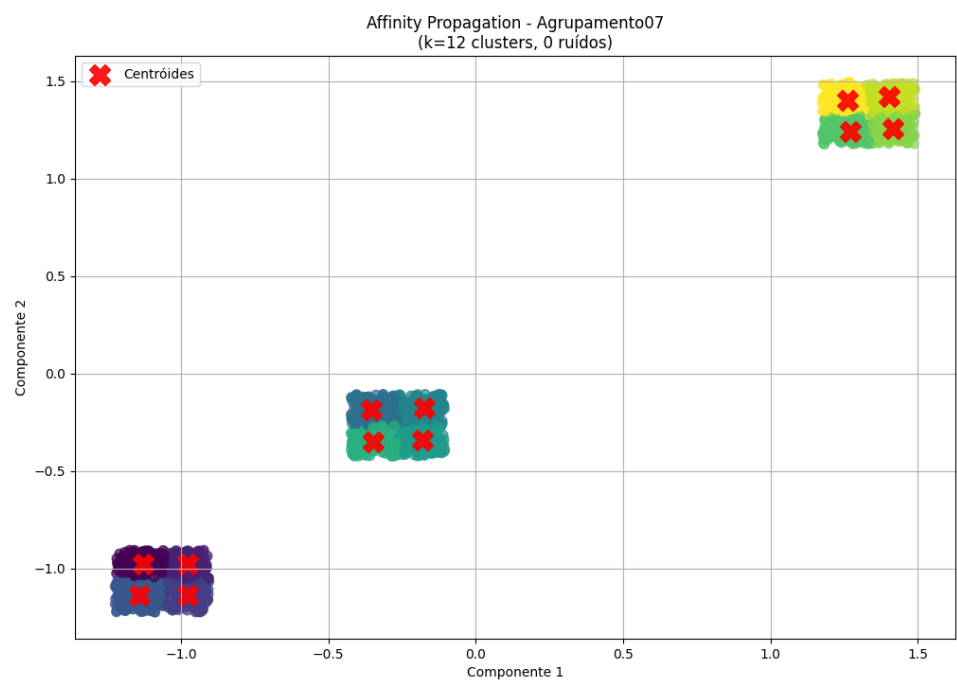
2.2) Mini Batch K-Means



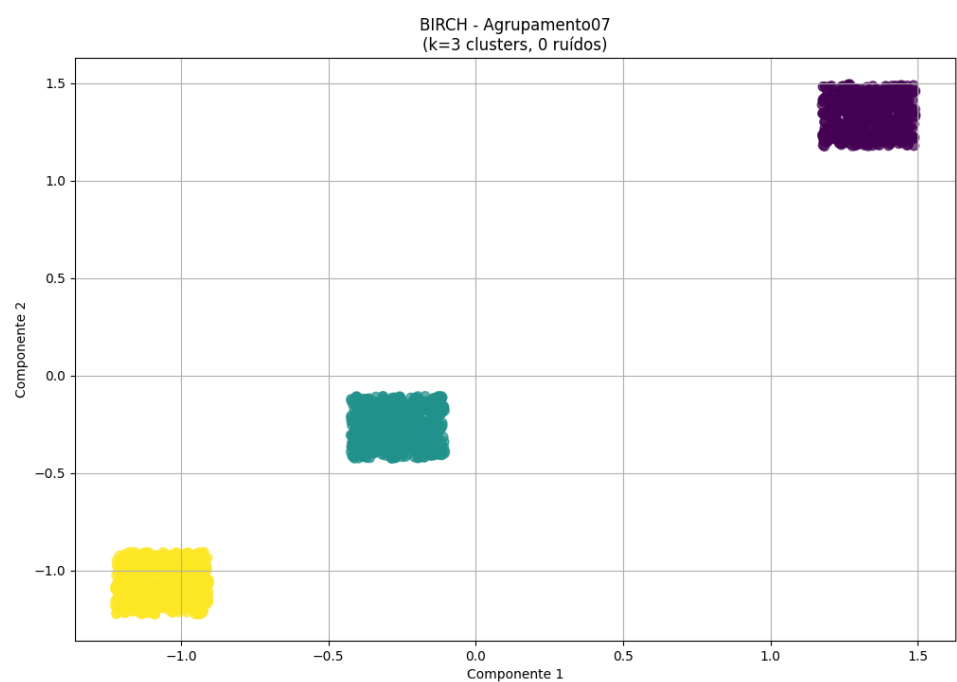
2.3) DBSCAN



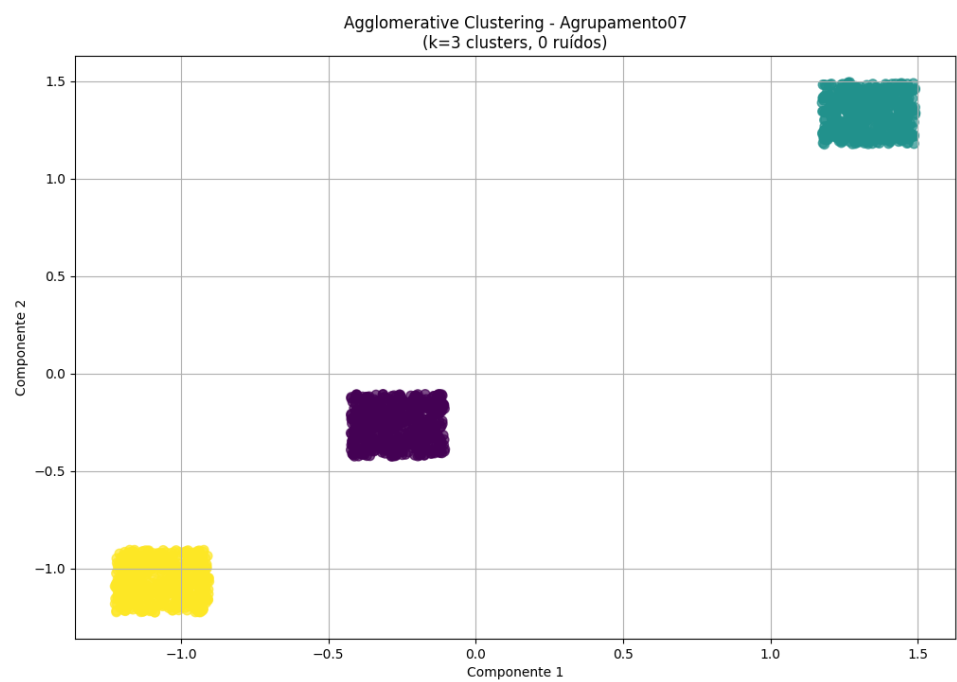
2.4) Affinity Propagation



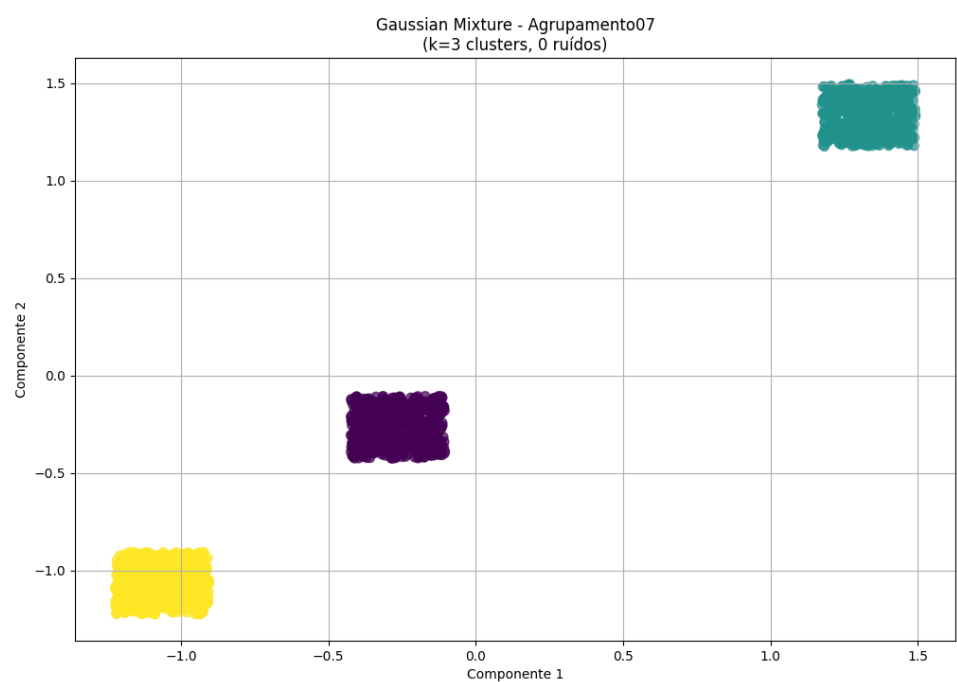
2.5) BIRCH



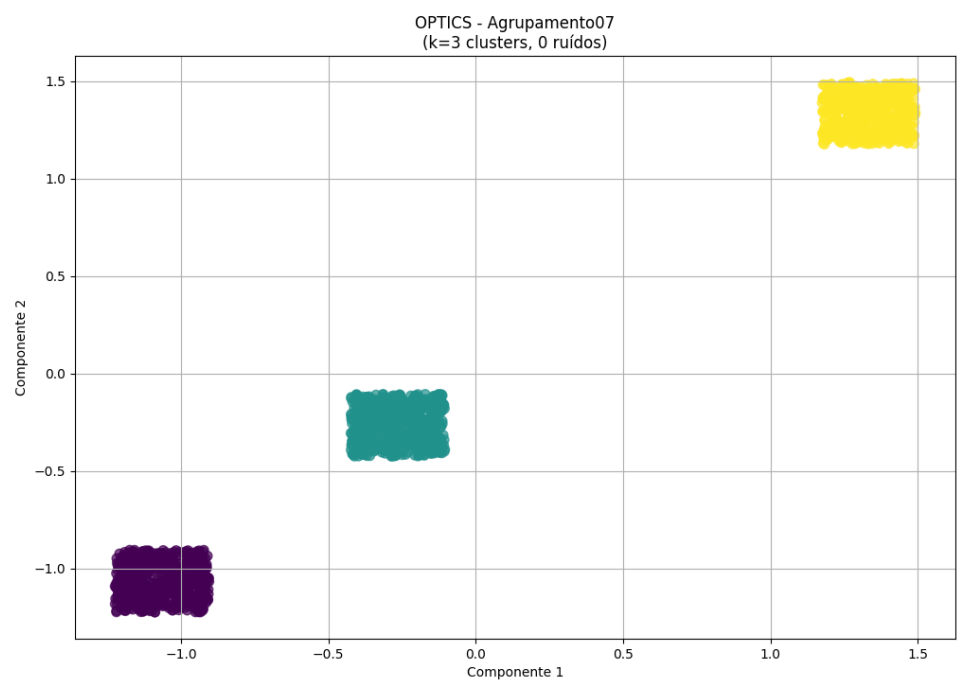
2.6) Agglomerative Clustering



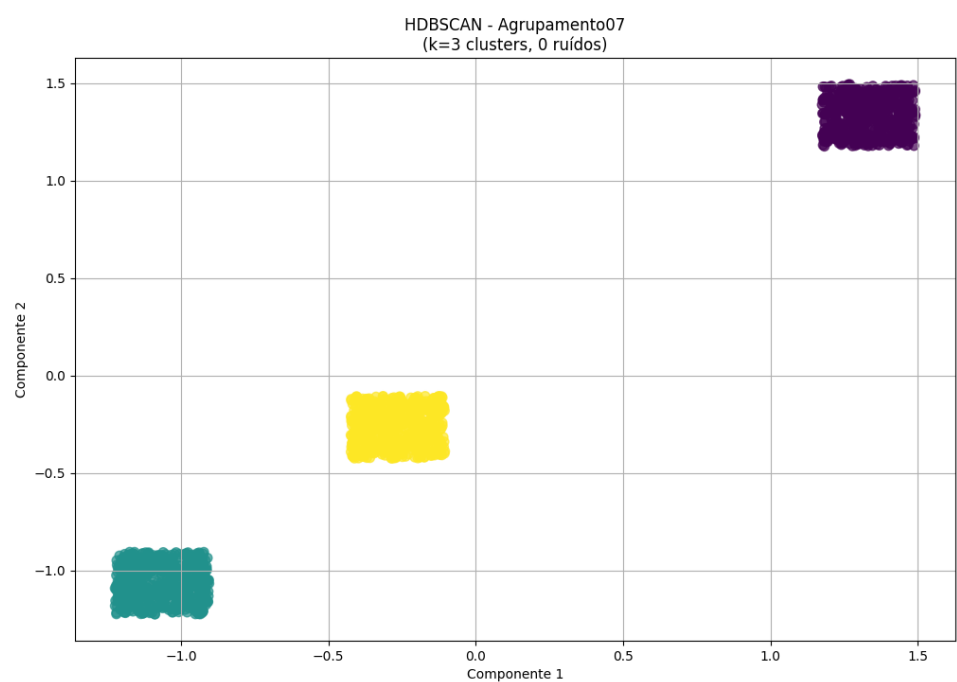
2.7) Gaussian Mixture



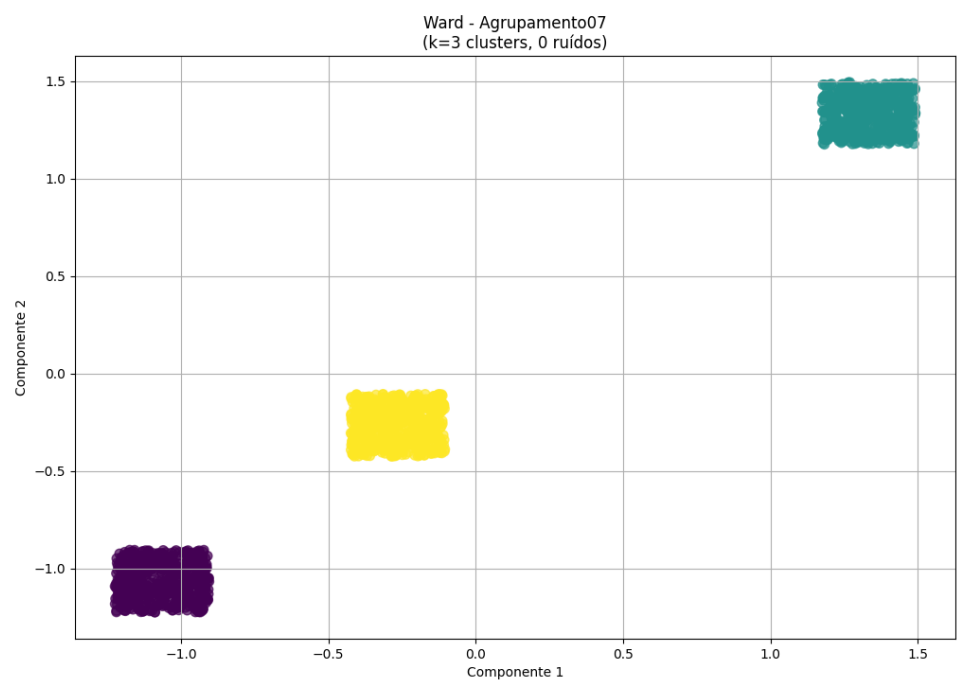
2.8) OPTICS



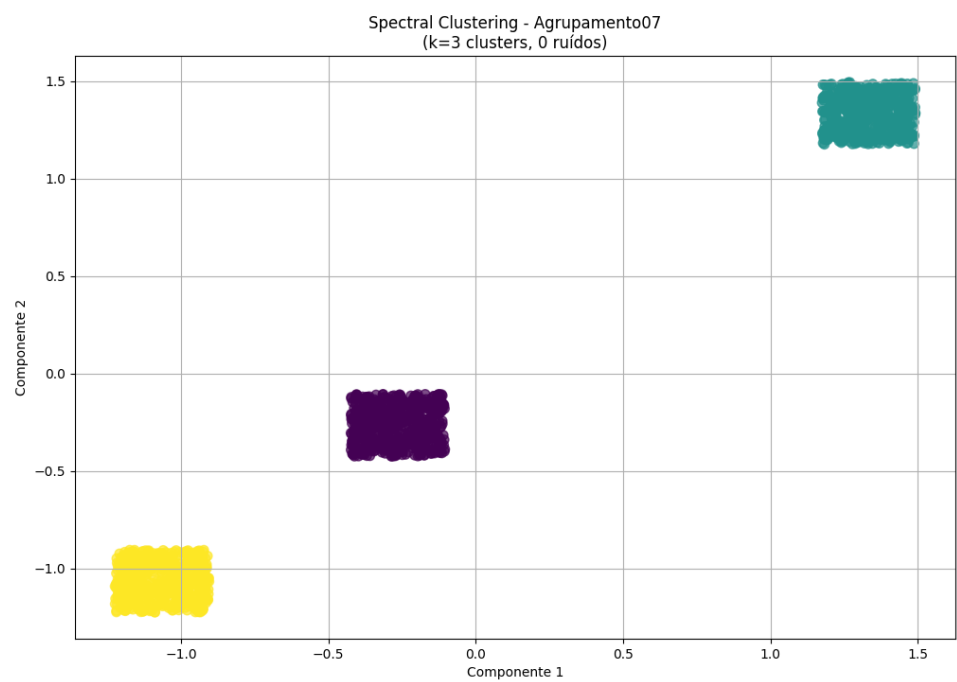
2.9) HDBSCAN



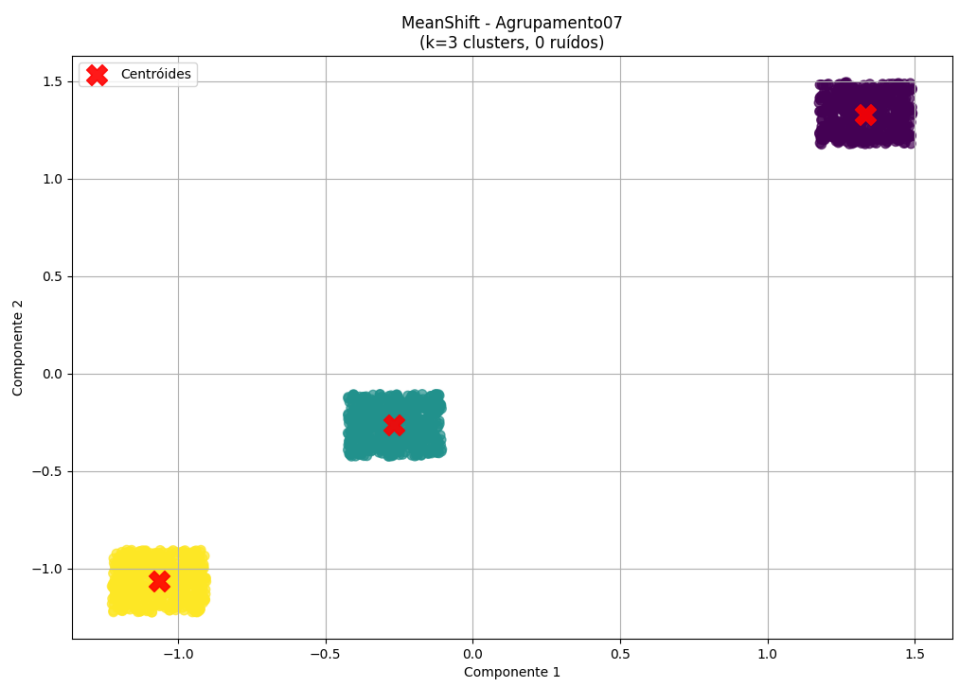
2.10) Ward



2.11) Spectral Clustering

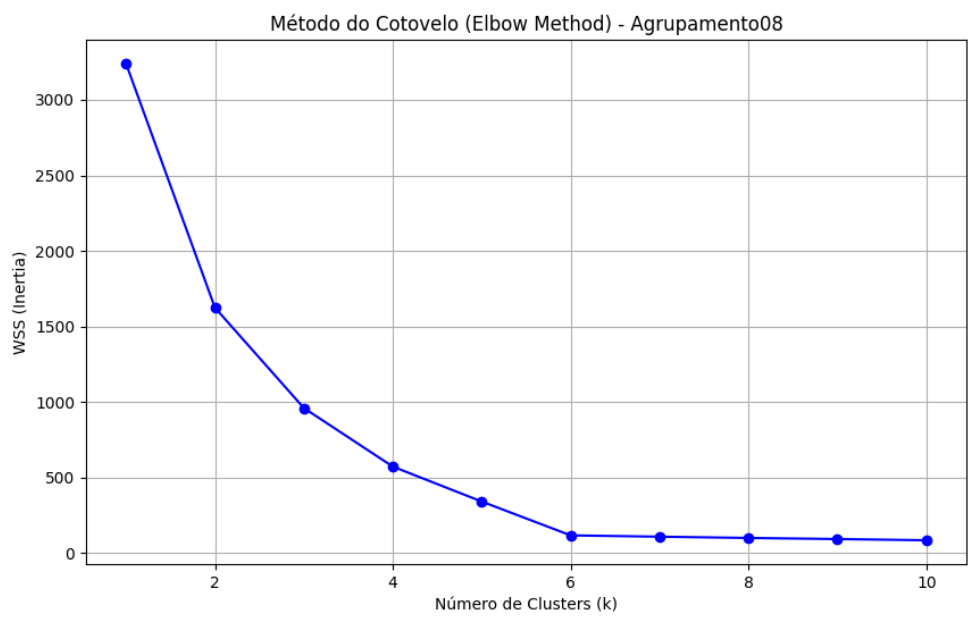


2.12) MeanShift

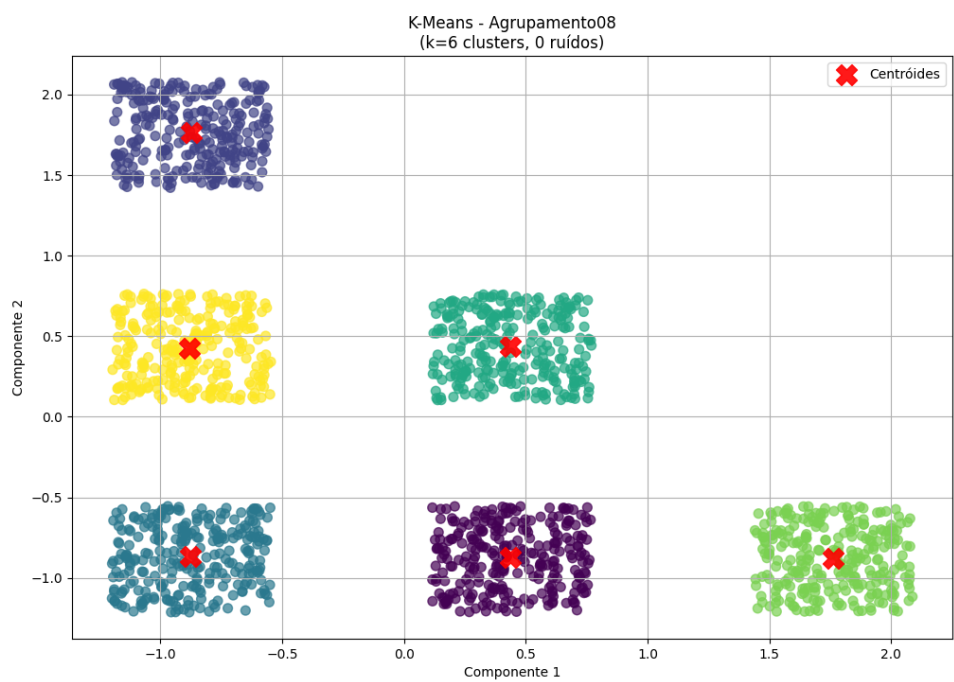


Problema 3: Agrupamento08.txt

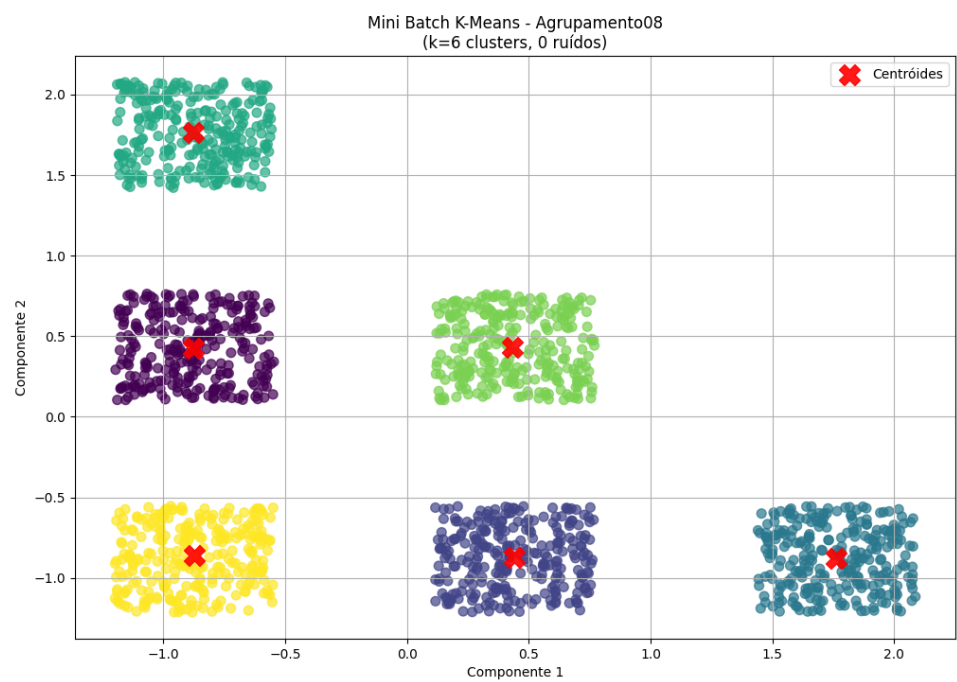
Metodo do Cotovelo (Elbow Method)



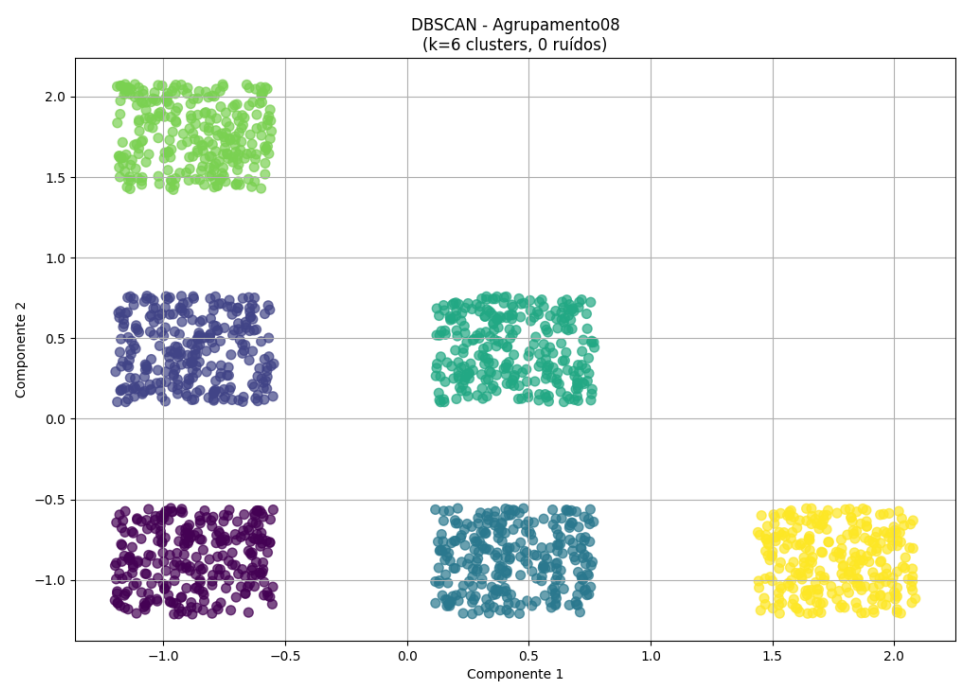
3.1) K-Means



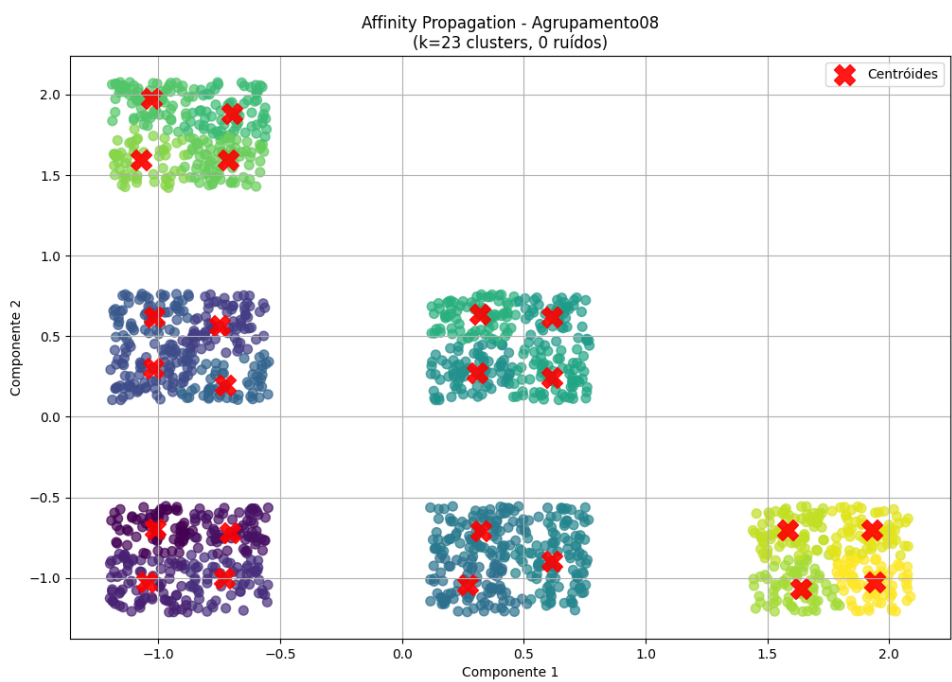
3.2) Mini Batch K-Means



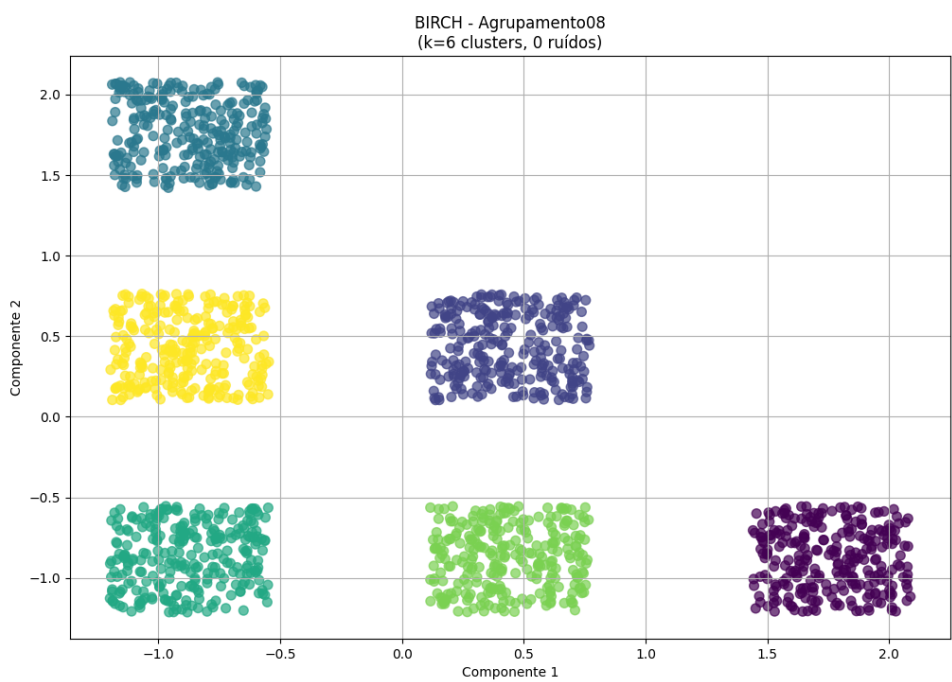
3.3) DBSCAN



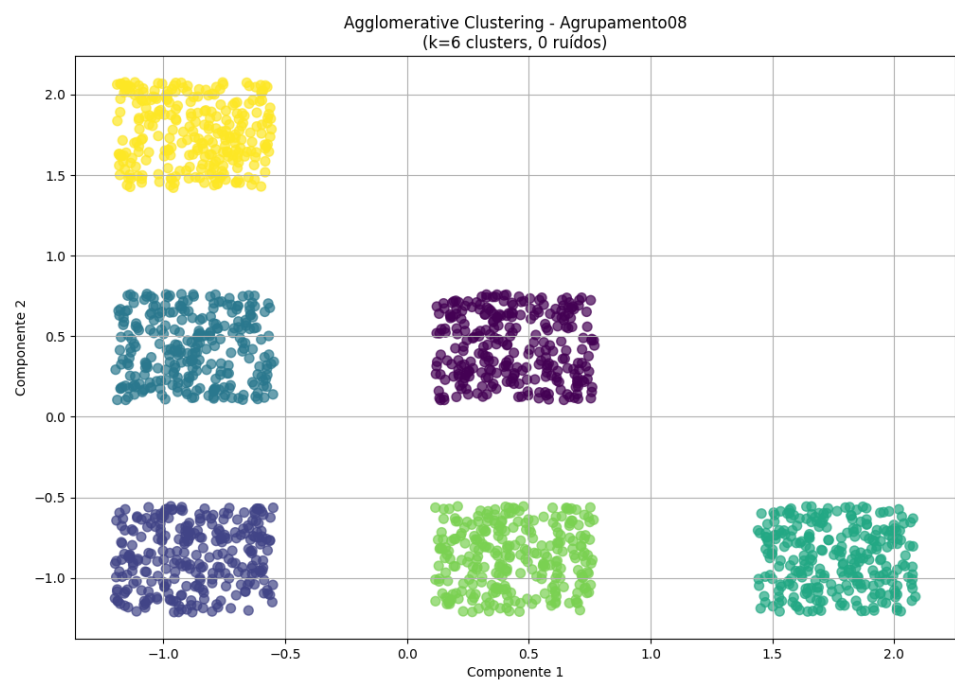
3.4) Affinity Propagation



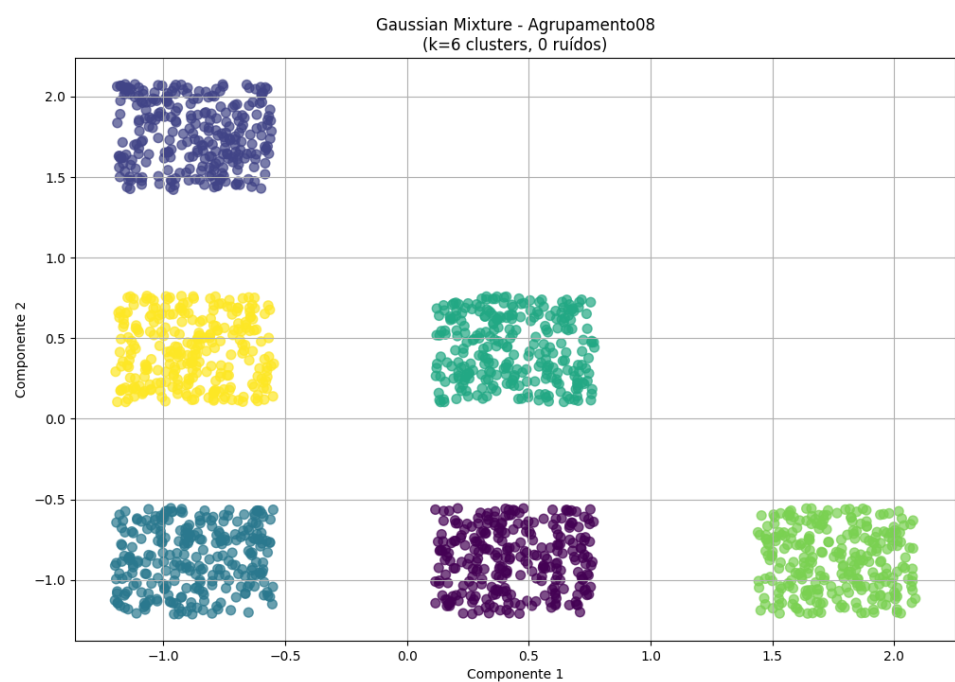
3.5) BIRCH



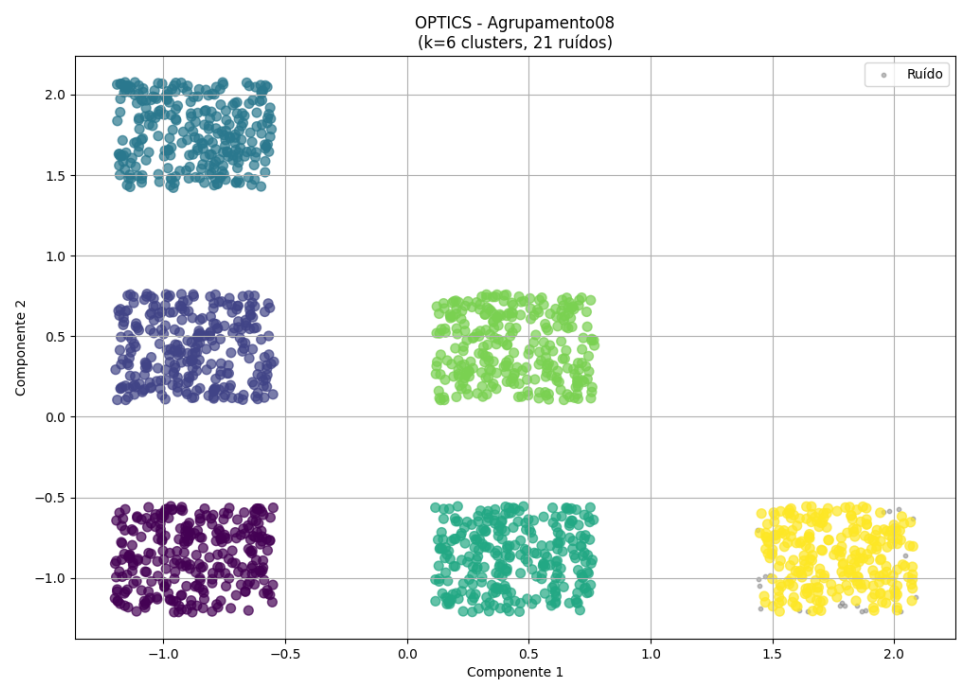
3.6) Agglomerative Clustering



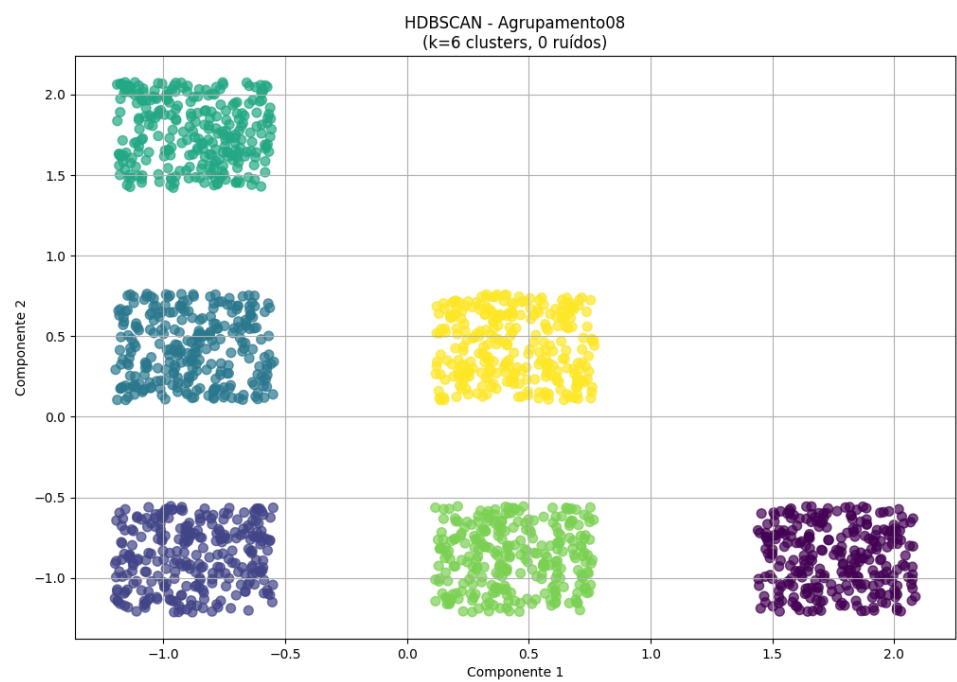
3.7) Gaussian Mixture



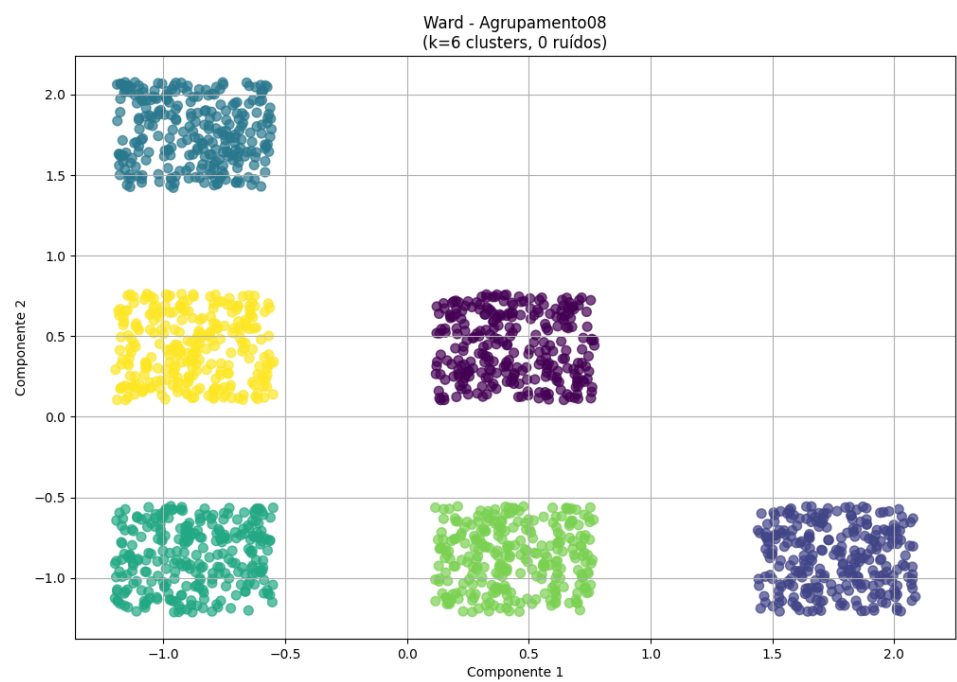
3.8) OPTICS



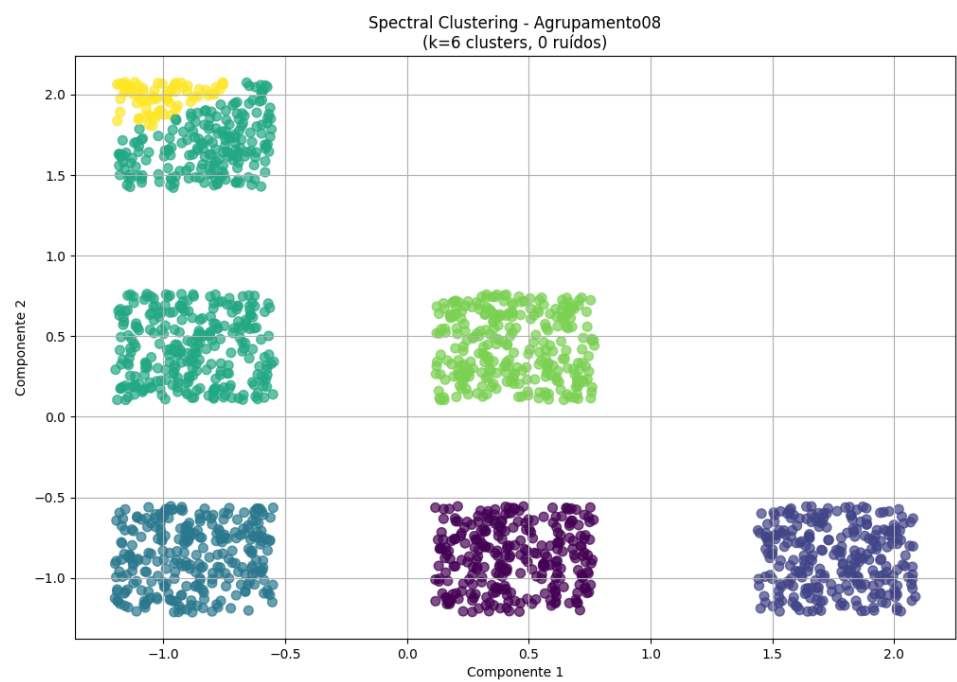
3.9) HDBSCAN



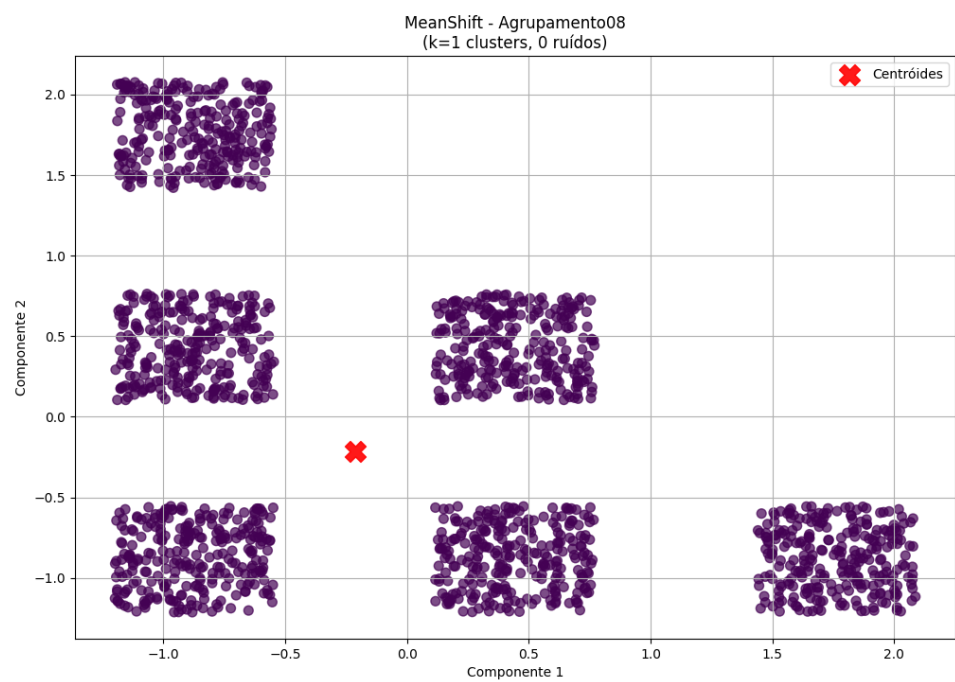
3.10) Ward



3.11) Spectral Clustering



3.12) MeanShift



Anexo: Código Fonte (resolver_trabalho.py)

```
# -*- coding: utf-8 -*-
import os
import warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import (
    KMeans,
    MiniBatchKMeans,
    DBSCAN,
    AffinityPropagation,
    Birch,
    AgglomerativeClustering,
    OPTICS,
    MeanShift,
    SpectralClustering,
)
from sklearn.mixture import GaussianMixture
from sklearn.neighbors import kneighbors_graph
from sklearn.utils._testing import ignore_warnings # Para ignorar UserWarnings
from sklearn.exceptions import ConvergenceWarning

# Tenta importar HDBSCAN, que é uma biblioteca separada
try:
    from hdbscan import HDBSCAN
except ImportError:
    print(
        "Aviso: HDBSCAN não instalado. "
        "Execute 'pip install hdbscan' para incluí-lo."
    )
    HDBSCAN = None # Define como None se não estiver disponível

# Importar kaleido não é necessário, mas ele precisa estar instalado
# para salvar imagens .png do Plotly
# import kaleido

# Ignorar avisos para uma saída mais limpa
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=ConvergenceWarning)

# --- Constantes de Diretório ---
# Assume que o script está dentro da pasta TP3
INPUT_DIR = "./content"
OUTPUT_DIR = "./result"

def plotar_grafico_cotovelo(X_scaled, title_suffix, output_basename):
    """
    Calcula e salva o gráfico do Método do Cotovelo (WSS).
    """
    print(f" Calculando WSS para o Método do Cotovelo ({title_suffix})...")
    wss = []
```

```

K_range = range(1, 11)

for k in K_range:
    kmeans_elbow = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_elbow.fit(X_scaled)
    wss.append(kmeans_elbow.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(K_range, wss, "bo-")
plt.xlabel("Número de Clusters (k)")
plt.ylabel("WSS (Inertia)")
plt.title(f"Método do Cotovelo (Elbow Method) - {title_suffix}")
plt.grid(True)
cotovelo_path = os.path.join(OUTPUT_DIR, f"{output_basename}_grafico_cotovelo.png")
plt.savefig(cotovelo_path)
plt.close()
print(f" Gráfico do Cotovelo salvo em: {cotovelo_path}")

def plotar_grafico_2d(X_plot, labels, title, output_path, centers=None):
    """
    Função auxiliar para criar e salvar gráficos 2D (PNG) com Matplotlib.
    """
    plt.figure(figsize=(12, 8))
    # 'labels == -1' é para ruído (comum em DBSCAN, OPTICS, HDBSCAN)
    # Damos a eles uma cor cinza e tamanho menor
    unique_labels = set(labels)

    if -1 in unique_labels:
        noise_mask = labels == -1
        plt.scatter(
            X_plot[noise_mask, 0],
            X_plot[noise_mask, 1],
            c="gray",
            s=10,
            alpha=0.5,
            label="Ruído",
        )
        # Plota os pontos não-ruído
        core_mask = ~noise_mask
        if np.any(core_mask): # Só plota se houver pontos core
            plt.scatter(
                X_plot[core_mask, 0],
                X_plot[core_mask, 1],
                c=labels[core_mask],
                s=50,
                cmap="viridis",
                alpha=0.7,
            )
    else:
        # Plota normalmente se não houver ruído
        plt.scatter(
            X_plot[:, 0],
            X_plot[:, 1],
            c=labels,
            s=50,
            cmap="viridis",
            alpha=0.7,

```

```

    )

# Plota centróides, se fornecidos
if centers is not None:
    plt.scatter(
        centers[:, 0],
        centers[:, 1],
        c="red",
        s=250,
        marker="x",
        alpha=0.9,
        label="Centróides",
    )

plt.title(title)
# Ajusta os labels dos eixos
if X_plot.shape[1] > 1:
    plt.xlabel("Componente 1")
    plt.ylabel("Componente 2")
else:
    # Caso fallback (improvável para 2D)
    plt.xlabel("Eixo X")
    plt.ylabel("Eixo Y")

if -1 in unique_labels or centers is not None:
    plt.legend()
plt.grid(True)
plt.savefig(output_path)
plt.close()

def plotar_grafico_3d(df_plot, labels_col, title, output_basename):
    """
    Função auxiliar para criar e salvar gráficos 3D interativos (HTML)
    e estáticos (PNG).
    """
    print(f"Gerando gráfico 3D: {title}...")

    # Cria uma cópia para evitar SettingWithCopyWarning
    df_plot_copy = df_plot.copy()

    # Converte labels para string para Plotly tratar cores discretas
    df_plot_copy[labels_col] = df_plot_copy[labels_col].astype(str)

    # Mapeia ruído para cinza
    color_map = {"-1": "grey"}

    # Renomeia colunas para o Plotly usar os nomes corretos
    col_names = list(df_plot.columns)
    x_col, y_col, z_col = col_names[0], col_names[1], col_names[2]

    fig = px.scatter_3d(
        df_plot_copy,
        x=x_col,
        y=y_col,
        z=z_col,
        color=labels_col,
        title=title,
    )

```

```

        color_discrete_map=color_map,
    )

camera_angle = dict(
    up=dict(x=0, y=0, z=1),      # Eixo Z é "para cima"
    center=dict(x=0, y=0, z=0),  # Olhando para a origem
    eye=dict(x=0.8, y=1.8, z=1.0) # <-- VALORES ALTERADOS AQUI
)
# -----

fig.update_layout(
    scene_camera=camera_angle, # <-- Linha adicionada
    margin=dict(l=20, r=20, t=40, b=20), paper_bgcolor="LightSteelBlue"
)

html_file_path = f"{output_basename}.html"
fig.write_html(html_file_path)

png_file_path = f"{output_basename}.png"
try:
    fig.write_image(png_file_path, width=800, height=600)
except Exception as e:
    print(f"  AVISO: Não foi possível salvar a imagem estática {png_file_path}.")
    print("  Certifique-se que 'kaleido' está instalado: pip install kaleido")
    print(f"  Erro: {e}")

def executar_e_plotar_algoritmos(
    X_scaled, X_plot, dataset_name, problem_prefix, k_ideal, is_3d=False
):
    """
    Executa todos os 12 algoritmos de clusterização e salva seus gráficos.
    """
    print(f"\nIniciando execução dos 12 algoritmos para {dataset_name}...")

    # X_plot pode ser um DataFrame (3D) ou np.array (2D)
    df_plot = None
    if is_3d:
        df_plot = X_plot.copy()

    # Parâmetros (alguns baseados no Colab do professor)
    params = {
        "quantile": 0.3,
        "eps": 0.5, # Valor padrão para DBSCAN, pode precisar de ajuste
        "min_samples": 10,
        "damping": 0.9,
        "preference": -200,
        "n_neighbors": 3,
        "hdbscan_min_cluster_size": 15,
        "hdbscan_min_samples": 3,
        "optics_min_samples": 10,
        "optics_xi": 0.05,
        "optics_min_cluster_size": 0.1,
    }

    # Ajusta parâmetros de DBSCAN e OPTICS para datasets menores
    # Os datasets 7 e 8 têm muito poucos pontos
    if X_scaled.shape[0] < 100:

```

```

print(" (Ajustando parâmetros para dataset pequeno)")
params["eps"] = 0.5 # Pode precisar de mais ajuste
params["min_samples"] = 3
params["optics_min_samples"] = 3
params["hdbscan_min_cluster_size"] = 2
params["hdbscan_min_samples"] = 2

# Conectividade para Ward e Agglomerative
connectivity = kneighbors_graph(
    X_scaled, n_neighbors=params["n_neighbors"], include_self=False
)
connectivity = 0.5 * (connectivity + connectivity.T)

# Largura de banda para MeanShift
bandwidth = 0.3 # Valor padrão, estimate_bandwidth pode ser lento
# Tentar estimar, mas usar padrão se falhar
try:
    # Reduzir o n_samples para datasets muito pequenos
    n_samples_bw = min(500, X_scaled.shape[0])
    # estimate_bandwidth pode ser lento, usar um subconjunto
    from sklearn.cluster import estimate_bandwidth
    bandwidth = estimate_bandwidth(X_scaled, quantile=params["quantile"], n_samples=n_samples_bw,
random_state=42)
    if bandwidth == 0: # Evitar bandwidth zero
        bandwidth = 0.3
    print(f" Largura de banda estimada (MeanShift): {bandwidth:.3f}")
except Exception as e:
    print(f" Aviso: estimate_bandwidth falhou ({e}), usando padrão 0.3.")
    bandwidth = 0.3

algoritmos = [
    ("K-Means", "kmeans", KMeans(n_clusters=k_ideal, random_state=42, n_init=10)),
    (
        "Mini Batch K-Means",
        "minibatch_kmeans",
        MiniBatchKMeans(n_clusters=k_ideal, random_state=42, n_init=10),
    ),
    (
        "DBSCAN",
        "dbscan",
        DBSCAN(eps=params["eps"], min_samples=params["min_samples"]),
    ),
    (
        "Affinity Propagation",
        "affinity",
        # Removido 'preference' para deixar o algoritmo decidir
        AffinityPropagation(damping=params["damping"], random_state=42),
    ),
    ("BIRCH", "birch", Birch(n_clusters=k_ideal)),
    (
        "Agglomerative Clustering",
        "agglomerative",
        AgglomerativeClustering(n_clusters=k_ideal),
    ),
    (
        "Gaussian Mixture",

```



```

        "gmm",
        GaussianMixture(n_components=k_ideal, random_state=42),
    ),
    (
        "OPTICS",
        "optics",
        OPTICS(
            min_samples=params["optics_min_samples"],
            xi=params["optics_xi"],
            min_cluster_size=params["optics_min_cluster_size"],
        ),
    ),
    (
        "Ward",
        "ward",
        AgglomerativeClustering(
            n_clusters=k_ideal, linkage="ward", connectivity=connectivity
        ),
    ),
    (
        "Spectral Clustering",
        "spectral",
        SpectralClustering(
            n_clusters=k_ideal, affinity="nearest_neighbors", random_state=42,
            n_init=10
        ),
    ),
    ("MeanShift", "meanshift", MeanShift(bandwidth=bandwidth, bin_seeding=True)),
]

```

Adiciona HDBSCAN apenas se foi importado com sucesso

```

if HDBSCAN:
    algoritmos.insert(
        8, # Insere antes do Ward
        (
            "HDBSCAN",
            "hdbscan",
            HDBSCAN(
                min_cluster_size=params["hdbscan_min_cluster_size"],
                min_samples=params["hdbscan_min_samples"],
                allow_single_cluster=True,
            ),
        ),
    )

```

```

for nome_amigavel, nome_arquivo, algoritmo in algoritmos:
    print(f" Executando ({nome_amigavel})...")

```

```

output_basename = os.path.join(OUTPUT_DIR, f"{problem_prefix}_{nome_arquivo}")

```

```

try:
    # Ajuste para GMM que não tem 'fit_predict'
    if hasattr(algoritmo, "fit_predict"):
        labels = algoritmo.fit_predict(X_scaled)
    else:
        algoritmo.fit(X_scaled)
        if hasattr(algoritmo, "predict"):
            labels = algoritmo.predict(X_scaled)

```

```

elif hasattr(algoritmo, "labels_"): # Para Agglomerative/Ward
    labels = algoritmo.labels_
else:
    print(f" Erro: Algoritmo {nome_amigavel} não tem 'predict' ou 'labels_'.")
    continue

# Ajuste para Affinity Propagation e K-Means que têm centróides
centers = None
if hasattr(algoritmo, "cluster_centers_"):
    # Centros estão no espaço escalado (X_scaled)
    # Se estamos plotando PCA (is_3d=False e X_plot != X_scaled),
    # precisaríamos transformar os centros para o espaço PCA.
    # O TP2 original simplificou não plotando centros no PCA.
    # Vamos seguir a regra: só plotar centros se X_plot == X_scaled

    # Para 3D (is_3d=True), X_plot é o original, X_scaled é escalado.
    # Não vamos plotar centros no 3D.
    if not is_3d and X_scaled.shape == X_plot.shape:
        centers = algoritmo.cluster_centers_
    else:
        centers = None # Não plotar em 3D ou PCA

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = np.sum(labels == -1) if -1 in labels else 0

title = (
    f"{nome_amigavel} - {dataset_name}\n"
    f"(k={n_clusters} clusters, {n_noise} ruídos)"
)

if is_3d:
    labels_col_name = f"{nome_arquivo}_labels"
    # Remove a coluna se ela já existir de uma rodada anterior
    if labels_col_name in df_plot.columns:
        df_plot = df_plot.drop(columns=[labels_col_name])

    df_plot[labels_col_name] = labels
    plotar_grafico_3d(df_plot, labels_col_name, title, output_basename)
else:
    # Para P2 e P3 (2D), X_plot = X_scaled
    output_png_path = f"{output_basename}.png"
    plotar_grafico_2d(X_plot, labels, title, output_png_path, centers=centers)

except Exception as e:
    print(f" !!! ERRO ao executar {nome_amigavel}: {e}")
    # Cria um gráfico de erro
    plt.figure(figsize=(12, 8))
    plt.text(0.5, 0.5, f"Falha ao executar {nome_amigavel}\n{e}",
             horizontalalignment='center', verticalalignment='center',
             fontsize=12, color='red', wrap=True)
    plt.title(f"{nome_amigavel} - {dataset_name}\n(FALHA NA EXECUÇÃO)")
    plt.savefig(f"{output_basename}_ERRO.png")
    plt.close()

print(f" {dataset_name} concluído.")

```

```

# ---
# PROBLEMA 1 (TP3): Agrupamento06.txt (3D)
# ---
def resolver_problema_1(input_dir, output_dir):
    print("\nIniciando Problema 1 (Agrupamento06.txt)...")
    file_path = os.path.join(input_dir, "Agrupamento06.txt")
    try:
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        print(f"ERRO: Arquivo não encontrado em {file_path}")
        return
    except Exception as e:
        print(f"ERRO ao ler {file_path}: {e}")
        return

    # X1, X2, X3
    X = df.iloc[:, 0:3]

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    plotar_grafico_cotovelo(X_scaled, "Agrupamento06", "p1")

    k_ideal_p1 = 4

    # Para P1 (3D), treinamos em X_scaled, mas plotamos o X original
    executar_e_plotar_algoritmos(
        X_scaled, X, "Agrupamento06", "p1", k_ideal_p1, is_3d=True
    )

# ---
# PROBLEMA 2 (TP3): Agrupamento07.txt (2D)
# ---
def resolver_problema_2(input_dir, output_dir):
    print("\nIniciando Problema 2 (Agrupamento07.txt)...")
    file_path = os.path.join(input_dir, "Agrupamento07.txt")
    try:
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        print(f"ERRO: Arquivo não encontrado em {file_path}")
        return
    except Exception as e:
        print(f"ERRO ao ler {file_path}: {e}")
        return

    # Real, Imag
    X = df.iloc[:, 0:2]

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    plotar_grafico_cotovelo(X_scaled, "Agrupamento07", "p2")

    k_ideal_p2 = 3

    # Para P2 (2D), plotamos os dados escalados (X_scaled)
    # (Seguindo o padrão do TP2 para problemas 2D)

```

```

    executar_e_plotar_algoritmos(
        X_scaled, X_scaled, "Agrupamento07", "p2", k_ideal_p2, is_3d=False
    )

# ---
# PROBLEMA 3 (TP3): Agrupamento08.txt (2D)
# ---
def resolver_problema_3(input_dir, output_dir):
    print("\nIniciando Problema 3 (Agrupamento08.txt)...")
    file_path = os.path.join(input_dir, "Agrupamento08.txt")
    try:
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        print(f"ERRO: Arquivo não encontrado em {file_path}")
        return
    except Exception as e:
        print(f"ERRO ao ler {file_path}: {e}")
        return

    # XX, YY
    X = df.iloc[:, 0:2]

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    plotar_grafico_cotovelo(X_scaled, "Agrupamento08", "p3")

    k_ideal_p3 = 6

    # Para P3 (2D), plotamos os dados escalados (X_scaled)
    executar_e_plotar_algoritmos(
        X_scaled, X_scaled, "Agrupamento08", "p3", k_ideal_p3, is_3d=False
    )

# --- Bloco de Execução Principal ---
if __name__ == "__main__":
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    os.makedirs(INPUT_DIR, exist_ok=True)

    print(f"Iniciando Terceiro Trabalho Prático...")
    print(f"Verificando arquivos de entrada em: {os.path.abspath(INPUT_DIR)}")
    print(
        "Certifique-se que 'Agrupamento06.txt', 'Agrupamento07.txt', 'Agrupamento08.txt' estão lá."
    )
    print(f"Resultados serão salvos em: {os.path.abspath(OUTPUT_DIR)}\n")

    resolver_problema_1(INPUT_DIR, OUTPUT_DIR)
    resolver_problema_2(INPUT_DIR, OUTPUT_DIR)
    resolver_problema_3(INPUT_DIR, OUTPUT_DIR)

    print("\n--- Processo Concluído ---")
    print(f"Todos os gráficos (PNG e HTML) foram salvos no diretório '{OUTPUT_DIR}'.")

```